ARCUS-X: ADAPTIVE RULE COMPLIANCE UNDER SEQUENTIAL EXECUTION FOR DIAGNOSING LONG-HORIZON LANGUAGE MODEL RELIABILITY

Anonymous authors
Paper under double-blind review

ABSTRACT

Long-horizon language-model evaluations often reduce performance to final-answer accuracy, obscuring failures that accumulate across dependent sequential transitions. We introduce ARCUS-X, an auditable benchmark and evaluation protocol for decomposing deterministic sequential execution into trajectory correctness, continuity, rule binding, horizon control, response validity, and excess generation. ARCUS-X uses procedurally generated toroidal-grid tasks with exact per-transition ground truth, controlled lexical and transition perturbations, raw-stream preservation, and deterministic post-run diagnostics. Across 13 models, ARCUS-X exposes substantial separation between aggregate trajectory quality and observed failure signatures. In the adaptively sampled runs, GPT-5-mini attains 85.3% step accuracy, 36.0% exact match, and 85.0% continuity, but 25.6% generation efficiency and 11.2% invalid responses. The valid-probe partitions differ sharply: state-tracking signatures account for 94.0% of valid Qwen 3 Coder 480B A35B probes and 84.3% of valid GLM-5 probes, while Mistral Large 2512 has 24.8% transition-rule signatures and Kimi K2.5 has 23.2% semantic-interpretation signatures. Horizon signatures, including overlong outputs, account for 49.4% and 56.1% of valid Mistral Large and Kimi probes, respectively. Sampled fracture behavior also varies by perturbation tier. Because AFS allocates horizons from observed performance, pooled aggregates describe the acquired support rather than a common uniform horizon distribution. ARCUS-X therefore provides a reproducible diagnostic record of explicit state-transition execution rather than a universal model ranking or measure of general long-horizon agency.

1 INTRODUCTION

As large language models (LLMs) are increasingly used for complex scientific and software tasks, reliability over long sequences has become an important evaluation problem. Existing evaluations such as GSM8K Cobbe et al. (2021), MMLU Hendrycks et al. (2020), and LiveBench Gu et al. (2024) primarily measure task-level correctness, while execution-based evaluations such as HumanEval Chen et al. (2021) test functional correctness. These evaluations are useful for determining whether a task succeeds, but they provide limited information about how reliability changes as dependent intermediate states accumulate across a sequence.

This becomes important when two models achieve similar final outcomes but fail in fundamentally different ways. One may gradually drift from the correct state, while another may preserve the state but fail to terminate correctly. Final-answer metrics can therefore hide behaviors such as state drift, incorrect intermediate transitions, horizon-dependent degradation, and failures to preserve constraints throughout execution.

Recent work has begun to study reliability across longer executions and through trajectory-level analysis. *Beyond pass@1* Khanal et al. (2026) examines reliability as task duration increases, while *Long-Horizon Agent Trajectory Attribution* Chen et al. (2026), AgentRewardBench Lù et al. (2025), Plan-RewardBench Wang et al. (2026), and KellyBench Grady et al. (2026) study long-horizon behavior through trajectories, rewards, attribution, and sequential decision making. ARCUS-X complements this work by focusing on a narrower unit of analysis: **sequential state-transition execu-**

tion. Rather than evaluating an entire agent environment, ARCUS-X uses procedurally generated deterministic tasks where each transition has deterministically computable ground truth, independent from model evaluation. This allows horizon and semantic perturbations to be controlled directly while measuring how reliably models maintain and transform state over time.

To address this, we introduce ARCUS-X as an auditable benchmark and evaluation protocol for long-horizon sequential execution. ARCUS-X evaluates the trajectory rather than only its final outcome, characterizing state preservation, rule binding, perturbation robustness, horizon control, sampled fracture behavior, validity, and output overhead. Its central object is diagnostic decomposition: similar aggregate accuracy can conceal different observable signatures, including state drift, rule misapplication, rule-binding errors, premature termination, and formatting failure. ARCUS-X is designed not to introduce trajectory evaluation itself, but to assign reproducible behavioral signatures under controlled transitions, perturbations, operational validity, and an explicit precedence policy. These labels describe benchmark behavior, not hidden model cognition.

This work makes the following contributions:

1. We introduce ARCUS-X, an auditable, procedurally generated benchmark and evaluation protocol for sequential state-transition execution reliability.
2. We develop trajectory-level metrics and a deterministic diagnostic taxonomy that distinguish state tracking, transition-rule application, semantic interpretation, horizon collapse, and formatting without reducing reliability to a single accuracy value.
3. We provide controlled lexical and transition perturbations that probe rule binding separately from transition-system execution, together with a bounded AFS protocol that directs evaluation toward threshold-relevant horizons without exhaustively evaluating every horizon.
4. We report an auditable evaluation workflow with deterministic seeded probes, independently validated ground truth, raw-stream preservation, resumable evaluation, experiment metadata, and explicit separation of operational from cognitive failures.

Code, evaluation artifacts, and reproducibility materials are available at `https://anonymous.4open.science/r/ARCUS-X`.

2 Related Work

Existing LLM evaluations such as GSM8K Cobbe et al. (2021), MMLU Hendrycks et al. (2020), and LiveBench Gu et al. (2024) primarily measure task-level correctness, while HumanEval Chen et al. (2021) evaluates functional correctness through execution-based tests. These benchmarks assess whether a task is completed, but they provide limited information about how reliability changes across dependent state transitions.

Recent work extends evaluation toward reliability and trajectory-level analysis. *Beyond pass@1* Khanal et al. (2026) studies reliability as a function of task duration, including reliability decay, variance amplification, and meltdown onset. *Long-Horizon Agent Trajectory Attribution* Chen et al. (2026), AgentRewardBench Lù et al. (2025), and Plan-RewardBench Wang et al. (2026) further examine trajectory-level behavior, attribution, and evaluation.

The Illusion of Diminishing Returns Sinha et al. (2026) isolates long-horizon execution by supplying the knowledge and plan needed for a task, and studies how reliability changes with execution length. *Reasoning or Reciting?* Wu et al. (2024) evaluates counterfactual task variants, including altered spatial conventions. ARCUS-X builds on these directions rather than claiming that execution-depth effects or rule perturbations are new in isolation.

RULER Hsieh et al. (2024) evaluates long-context capabilities through retrieval, multi-hop tracing, and aggregation tasks. LongMemEval Wu et al. (2025) evaluates long-term conversational memory, including information extraction, multi-session and temporal reasoning, knowledge updates, and abstention. LoCoMo Maharana et al. (2024) examines long-term conversational memory through multi-session interactions and associated reasoning tasks.

Broader agent benchmarks instead evaluate success in naturalistic, multi-step environments. SWE-bench Jimenez et al. (2024) evaluates repository-level issue resolution, WebArena Zhou et al. (2024) evaluates web-agent behavior, and AgentBench Liu et al. (2024) spans multiple interactive environments. These settings deliberately combine planning, tool use, observation, and environment-specific uncertainty. ARCUS-X is not a substitute for them. Rather, it removes those factors to test whether a model follows an explicitly specified deterministic transition system. This controlled scope permits exact trajectory-level attribution, but does not establish general long-horizon agent capability or reliability across long-horizon agents. KellyBench Grady et al. (2026) and related sequential decision-making environments provide additional tests of extended decision making.

ARCUS-X addresses a narrower evaluation objective than these long-context and memory benchmarks: the reliability of executing explicitly specified deterministic state transitions over dependent sequences. Rather than primarily testing retrieval or memory across extended contexts, ARCUS-X evaluates intermediate states against a computer-generated reference trajectory. Its contribution lies in jointly characterizing transition correctness, temporal continuity, exact trajectory agreement, horizon compliance, operational validity, and generation overhead under controlled rule perturbations. Long-horizon degradation and trajectory evaluation are not claimed as novel in themselves; the benchmark provides an auditable decomposition of observable sequential execution behavior.

3 METHODOLOGY

ARCUS-X isolates long-horizon sequential execution reliability under controlled and deterministic perturbations.

3.1 EXPERIMENTAL DESIGN

Within ARCUS-X, the base unit of evaluation is a *probe*: a deterministic task instance defined by its initial coordinate, grid dimensions, transition rules, ordered action sequence, and horizon (z), the number of generated actions; the evaluated trajectory contains z coordinates including the initial state. A useful visualization is to view each transition as a link in a chain, with larger horizons forming longer chains. ARCUS-X computes an authoritative ground-truth trajectory from the environment's transition rules and withholds it from the model.

3.2 PROCEDURAL GENERATION OF PROBES

ARCUS-X uses seeded procedural generation: the same seed and configuration regenerate the same probe. Unless a grid-size sweep is enabled, width W and height H are sampled independently and uniformly from $[5, 12]$; initial coordinates and action sequences are deterministically sampled from the same probe seed. Horizon and perturbation tier are controlled separately, and every probe contains exactly the requested number of transitions.

3.3 TRANSITION RULES AND ENVIRONMENT

ARCUS-X's environment is based on a grid defined as the formal state space. The state is represented as a coordinate $[x, y]$. The grid is two-dimensional, composed of a width, *W*, and height, *H*. The environment is toroidal; hence, crossing an edge wraps to the opposite side.

Each action corresponds to its labeled deterministic transition $(\Delta x, \Delta y)$. At each step, the model must first identify the action, retrieve its associated transition, apply it to the current state, and finally wrap the resulting coordinate. This wraparound behavior is implemented using a modulo operation. It is a sequential task because the next state entirely depends on the previous state.

$$x_{t+1} = (x_t + \Delta x_t) \mod W$$
$$y_{t+1} = (y_t + \Delta y_t) \mod H$$

Coordinates are indexed from 0 to $W - 1$ along the horizontal axis, and likewise from 0 to $H - 1$ along the vertical axis.

3.4 PERTURBATION REGIMES

ARCUS-X perturbs the model-facing task along a four-level categorical *tier* axis (Tier 0–3). These tiers are the active perturbations analyzed here. Stored runs also contain a legacy semantic-gravity value g, but in the main runner g neither alters the prompt nor selects the tier; all four tiers are crossed with each configured g. We therefore interpret results by tier, not g as an independent semantic-difficulty variable.

1. Tier 0 provides the baseline, in which there are standard directional labels and $(\Delta x, \Delta y)$ rules. Tier 0 establishes baseline state-tracking performance.
2. Tier 1 is a semantic disruption tier. It renames action labels to pseudowords; however, the underlying transition structure remains unchanged. This tests whether the model can bind arbitrary labels to the explicitly given rules rather than relying on familiar words such as "north" or "south."
3. Tier 2 is an attribute inversion tier, in which each transition is transformed as follows:
   $(\Delta x, \Delta y) \rightarrow (\Delta x, -\Delta y)$
   The ground truth is recomputed using the transformed rules. This tests whether the model follows the provided transition system rather than its prior assumptions about spatial directions.
4. Tier 3 acts as a coordinate transformation tier. It transforms each transition using a 90° rotation:
   $(\Delta x, \Delta y) \rightarrow (-\Delta y, \Delta x)$
   This tests whether the model can maintain the state-transition procedure when the spatial behavior conflicts with ordinary directional intuition. Ground truth is recomputed using the transformed transition rules.

The prompt contains the assigned labels and rules, grid dimensions, initial state, action sequence, question, and required output format; it omits g, hidden answers, and difficulty metadata. Stored runs use $g \in \{0.5, 3.0, 15.0\}$ crossed with all four tiers; because g is inert in this sweep, these values function as replicated configuration strata rather than semantic perturbations. Fixed calibration probes at $g = 0.0$, $z = 3$, Tier 1 are parsing/environment checks and are excluded from reported metrics.

3.5 EVALUATION

We implement a native evaluation harness for ARCUS-X that separates probe generation, deterministic environment execution, model inference, response parsing, trajectory scoring, diagnostic classification, and adaptive fracture analysis. Model outputs are evaluated against independently computed ground-truth trajectories.

ARCUS-X records the model's trajectory, raw response, and inference metadata, including latency, output tokens, completion status, and provider/API errors. These measurements are independent of trajectory correctness, allowing execution, formatting, and efficiency failures to be distinguished from state-tracking errors. Responses are classified as valid trajectory, malformed recoverable output, empty output, parser failure, provider failure, or runtime/exception failure.

The parsed trajectory is compared against ground truth to calculate transition accuracy, exact trajectory match, and transition continuity. The evaluated reference contains the provided initial state and the first $z - 1$ action results: $L_{\mathrm{GT}} = z$. The final action remains in the prompt but its result is not scored. A response with $z + 1$ coordinates is overlong under this criterion even if it correctly executes every action. The predicted length L_{pred} is the number of parsed coordinates, with overlap $L_{\mathrm{overlap}} = \min(L_{\mathrm{pred}}, L_{\mathrm{GT}})$. Partial trajectories are scored rather than discarded. The initial state is retained for exact-match and structural-length checks but is excluded from transition accuracy and transition continuity because it is provided in the prompt.

ARCUS-X calculates the following raw measurements:

1. Output tokens: total generated tokens obtained from provider metadata or estimated, with source and benchmark confidence saved in raw output files.

2. Output tokens per second.
3. Correct transitions per second.

ARCUS-X uses multiple master seeds generated via a TRNG utilizing atmospheric noise Haahr (1998). Main tables report pooled probe-level means over the observations actually acquired by AFS. Because AFS may evaluate different horizon sets for different models and conditions, these pooled quantities are descriptive summaries of the observed support, not estimates under a shared uniform horizon distribution. Appendix A.6 reports the population mean and standard deviation of CRI across the five master seeds; raw artifacts retain per-seed CRI and fracture depth. We do not claim statistical significance for between-model differences.

3.6 Failure Taxonomy

The taxonomy is a deterministic post-scoring diagnostic, not an inference about a model's internal cognition. Its implementation is the released `classify_trajectory` and `_attribute_mechanism` functions in `arcus/analysis/taxonomy.py`; the full category definitions, precedence rules, output-validity split, and serialized evidence are specified in Appendix A.1. We therefore refer to its outputs as *failure signatures* or diagnostic categories rather than causal mechanisms. The label *none* requires an operationally valid, complete trajectory whose coordinates exactly match the canonical reference path; perfect Step Accuracy alone is insufficient. Invalid responses never enter the trajectory taxonomy, and valid failures that cannot be mapped are retained as *unknown/unmapped*. Category frequencies should be interpreted together with the stated precedence rules, including conservative fallback assignments, rather than as direct measurements of latent cognitive causes.

3.7 Metrics

The primary measures of sequential execution reliability are:

1. **Step Accuracy**

$$\text{Step Accuracy} = \frac{\text{number of correctly reproduced post-initial states}}{z-1}.$$

   Comparison is performed at aligned post-initial positions over the overlapping region. The initial state is not counted. At $z = 1$, no transitions are scored; the implementation records zero rather than interpreting this as evidence about transition execution.

2. **Exact Match**, which is binary per probe and requires identical coordinates and trajectory length.
3. **Continuity**, defined as the longest correct transition prefix:

$$\text{Continuity} = \frac{\text{longest correct post-initial prefix}}{z-1}.$$

First divergence is reported as a diagnostic rather than a primary aggregate metric. It records the 1-based position of the first mismatch; if the overlapping region is correct but the lengths differ, divergence is assigned to the first position beyond the overlap.

ARCUS-X also measures horizon and output behavior:

1. **Horizon Compliance**

$$\text{HC} = 1 - \frac{|L_{\text{pred}} - z|}{\max(L_{\text{pred}}, z)}.$$

2. **Generation Bloat Index**

$$\text{GBI} = \frac{\max(0, T_{\text{actual}} - T_{\text{expected}})}{T_{\text{expected}}},$$

   where T_{actual} is the completion-token count. The deterministic completion allowance is $T_{\text{expected}} = 10z + 10$: ten tokens for each of the z coordinate pairs plus a ten-token formatting allowance. GBI therefore measures *excess generation relative to this fixed benchmark*

allowance; it is not a tokenizer-invariant minimum-description-length estimate. Provider usage is used when available; otherwise token counts fall back, in order, to an official tokenizer, a compatible tokenizer, or a documented deterministic heuristic. Token source and confidence are retained per probe.

3. **Generation Efficiency** GE $= \frac{1}{1+\text{GBI}}$ per probe. Reported GE and GBI columns are each arithmetic means of their respective per-probe values; consequently, mean GE is not generally equal to $1/(1 + \text{mean GBI})$.

We additionally report the Composite Robustness Index (CRI) as a secondary, researcher-specified composite of trajectory fidelity, horizon robustness, perturbation robustness, and generation efficiency. Its formulation and fixed weights are provided in Appendix A.5; primary claims in this paper rely on the decomposed measurements rather than CRI alone.

ARCUS-X also reports system-level efficiency using latency. Latency is total wall-clock response time and is treated as a systems-level diagnostic because it depends on batching, deployment conditions, and provider infrastructure: ALE $= \text{Accuracy}/\text{Latency}_{\text{seconds}}$.

3.8 Adaptive Fracture Search

Adaptive Fracture Search (AFS) is a bounded, reproducible, threshold-directed allocation protocol that acquires observations near horizons where valid-response step accuracy falls below a 50% floor without scanning every integer horizon.

The search uses accuracy over all z action transitions, whereas the reported trajectory metrics evaluate the first $z - 1$ transitions. At each horizon z, cognitive step accuracy is computed over operationally valid trajectories: $A(z) = \frac{1}{|V_z|} \sum_{i \in V_z} a_i$,

where V_z is the set of analyzable probes and a_i is the step accuracy of probe i. Operational failures are excluded from $A(z)$ and reported separately. Accordingly, $A(z)$ is conditional accuracy given an analyzable trajectory; it should not be interpreted as end-to-end success when validity itself declines.

AFS evaluates midpoint horizons with a bisection-style controller and expands its upper bound while performance remains above the floor. The controller is an allocation heuristic, not a monotonic-capacity model: recoveries remain visible in the sampled curve and are handled post hoc. Invalid or infrastructure-failed responses are excluded from the batch mean; a batch with no valid observations is operationally insufficient evidence. Evaluated probes are cached and reused.

The live-search value z^* is a below-floor horizon encountered by the allocation procedure, or a censored ceiling outcome when none is observed; it is not guaranteed to be the first failing horizon. A separate forward-look-ahead analysis reports *sampled structural fracture*. Both operate on adaptively sampled points, so fracture values are condition-specific diagnostics of the acquired curve rather than intrinsic capacities. The configured ceiling is $z = 150$.

4 Results

4.1 Benchmark Performance

We evaluated ARCUS-X on 13 models and focus the detailed analysis on the five representative models in Table 1; the Appendix reports all 13. The paper's 13-model reporting cohort is preserved. The checkout also contains a Minimax M2.5 raw log; it is retained in the correction-package analyses and provenance but not silently added to the originally reported cohort. The representative set uses fixed Generally Available model versions: GPT-5-mini (Microsoft Azure Foundry, version 2025-08-07), Kimi K2.5 (AWS Bedrock), GLM-5 (AWS Bedrock), Qwen 3 Coder 480B A35B (AWS Bedrock), and Mistral Large 2512 (Mistral AI API). Runs used five probes per evaluated batch, five master seeds, and temperature 0 or the lowest allowed value. Because AFS evaluates model- and condition-dependent horizon sets, completed execution counts and horizon mixtures differ across runs; the exact matrix is retained in each manifest and raw artifact. Unless explicitly conditioned on a shared horizon, the pooled values below therefore compare the observed AFS-supported samples and should not be read as estimates over an identical horizon distribution. Within the observed AFS-supported pools, GPT-5-mini has the highest step accuracy, exact match, and

Table 1: Overall ARCUS-X trajectory-level performance over all recorded non-calibration probes. Asterisks indicate response validity below 75%. Validity denotes the proportion of probes yielding cognitively analyzable responses.

Model	Step Acc.	Exact Match	Continuity	Horizon Compliance	Validity
GPT-5-mini	85.3%	36.0%	85.0%	87.7%	88.8%
Kimi K2.5	59.2%	0.1%	53.9%	77.0%	94.8%
GLM-5	56.2%	1.0%	51.3%	98.4%	100.0%
Qwen 3 Coder 480B A35B	49.7%	1.2%	43.1%	98.3%	100.0%
Mistral Large 2512	56.0%	0.4%	53.0%	45.2%	96.9%

continuity of the five representative runs: 85.3% step accuracy, 36.0% exact match, and 85.0% continuity. The other four show 49.7–59.2% step accuracy and 0.1–1.2% exact match. GLM-5 and Qwen 3 Coder 480B A35B also show that high horizon compliance can coexist with lower trajectory correctness. These are run-level descriptive comparisons, not a controlled ranking over matched horizons.

4.2 Horizon-Dependent Degradation and Fracture

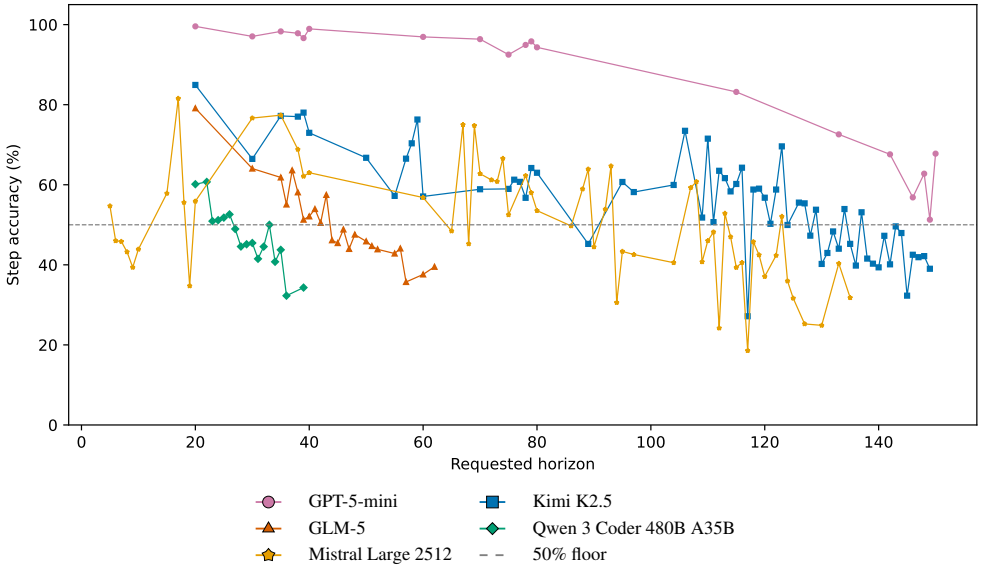


Figure 1: Per-horizon step accuracy over all recorded non-calibration probes for the five representative models, including invalid responses. Each sampled point is drawn with a model-specific marker (circle, square, triangle, diamond, or star) in addition to its color, so the curves remain distinguishable in grayscale.

Figure 1 shows the observed per-horizon step-accuracy profiles. The curves are non-monotonic, which is precisely why we treat AFS as threshold-directed sampling rather than an estimator of a monotone capacity boundary. GPT-5-mini remains high across most sampled horizons with a dip near $z \approx 70$; Kimi K2.5 shows an early dip followed by lower but comparatively stable sampled accuracy; GLM-5 declines more sharply; Qwen 3 Coder 480B A35B declines more gradually; and Mistral Large 2512 recovers at intermediate horizons before declining again. Lines connect observations for readability only. Point density and horizon coverage are nonuniform because AFS allocates probes adaptively (Appendix A.4), so the figure supports descriptive diagnosis of sampled behavior rather than interpolation between points or a monotonic-capacity claim.

Table 2 shows that sampled structural fracture is strongly condition-dependent rather than an invariant model property. GPT-5-mini has no post-hoc fracture in the sampled curves, whereas Kimi K2.5 has late sampled fractures in Tier 0 and Tier 2 but an earlier one in Tier 3. Mistral Large 2512 makes

Table 2: Post-hoc structural fracture depth by perturbation tier for the five representative models. Accuracy is pooled over all recorded non-calibration probes, including invalid responses. Values are the smallest sampled horizon below the 50% step-accuracy floor that is not followed by a recovery within the next two sampled horizons; 'N/A' means that no such sampled point was found.

Model	Tier 0	Tier 1	Tier 2	Tier 3
GPT-5-mini	N/A	N/A	N/A	N/A
Kimi K2.5	145	N/A	130	55
GLM-5	57	44	56	30
Qwen 3 Coder 480B A35B	34	30	29	23
Mistral Large 2512	9	5	94	6

the non-monotonicity especially clear: its Tier-2 sampled fracture is late despite much earlier values in the other tiers. The recovery rule operates on the next two *sampled* horizons, whose spacing is not uniform, so these values describe the acquired curves rather than a fixed-width horizon neighborhood. The live AFS window was capped at $H = 150$; reaching that cap without a below-floor midpoint is recorded separately as a saturated, right-censored result.

4.3 Semantic Robustness

As described in Section 3.4, the benchmark runner evaluates all four perturbation tiers at each stored gravity configuration. Table 3 reports per-tier step accuracy pooled over the observations acquired for each tier. Because AFS may sample different horizon mixtures across tiers, these values establish that the model-facing transformations produce distinct observed profiles, but they do not by themselves isolate a causal tier effect at matched horizons.

Table 3: Per-tier step accuracy for the five representative models. Tier 0 is baseline; Tier 1 renames action labels only; Tier 2 inverts the y-delta; Tier 3 rotates transitions $90°$.

Model	Tier 0	Tier 1	Tier 2	Tier 3
GPT-5-mini	88.0%	83.7%	87.5%	82.2%
Kimi K2.5	63.1%	75.8%	65.2%	32.9%
GLM-5	64.5%	58.7%	60.3%	41.3%
Qwen 3 Coder 480B A35B	56.9%	49.7%	53.7%	38.4%
Mistral Large 2512	56.1%	48.6%	58.6%	60.5%

The observed tier profiles are not ordered uniformly across models. GPT-5-mini spans 5.8 points between its highest and lowest pooled tier values. Kimi K2.5 and GLM-5 are substantially lower in Tier 3 than Tier 0 (30.2 and 23.2 points), while Qwen 3 Coder 480B A35B differs by 18.5 points. Mistral Large 2512 instead has its lowest pooled accuracy in Tier 1 and its highest in Tier 3, and Kimi K2.5 is higher in Tier 1 than Tier 0. These results argue against treating perturbation robustness as a single ordinal scalar: lexical relabeling and transition-altering transformations can produce different signatures for the same model. Because horizons are not matched by construction, we describe these as pooled tier associations rather than attributing the differences uniquely to the perturbation itself. Appendix A.7 reports all 13 models and the CRI decomposition.

4.4 Generation Efficiency and Failure Modes

Aggregate trajectory accuracy does not capture output overhead. We therefore report Generation Efficiency (GE), derived from GBI, as a benchmark-relative excess-generation score and Correct Steps per Second (CSS) as a secondary systems measure; latency and output-token throughput remain supporting diagnostics (Section 3.7).

ARCUS-X separates correctness from output overhead. GPT-5-mini combines high step accuracy with low GE, whereas GLM-5 and Qwen 3 Coder 480B A35B have near-100% GE but lower step accuracy. GE measures generation relative to the fixed benchmark allowance: 100% means only that output did not exceed that allowance, not tokenizer-independent optimality. CSS remains a provider- and deployment-dependent systems diagnostic and does not enter trajectory, fracture, GBI, or CRI scoring.

Table 4: Generation efficiency measurements for the five representative models.

Model	Step Accuracy	GBI	Generation Efficiency	Correct Steps/sec
GPT-5-mini	85.3%	4.25	25.6%	1.922
Kimi K2.5	59.2%	1.12	55.9%	2.406
GLM-5	56.2%	0.00	100.0%	4.626
Qwen 3 Coder 480B A35B	49.7%	0.02	99.4%	6.786
Mistral Large 2512	56.0%	4.49	23.8%	1.127

Table 5 reports the complete diagnostic partition for the representative models. Taxonomy percentages use valid probes as their denominator and include *None* (a fully correct trajectory) and *Unknown / Unmapped*; validity is reported separately over all completed probes. Every row's taxonomy columns sum to approximately 100% after rounding, and invalid responses are not assigned a trajectory-level failure signature. Because the categories are precedence-based, their relative frequencies describe how the deterministic classifier partitions observed trajectories rather than independently validated latent mechanisms.

Table 5: Post-run validity and taxonomy partition for the five representative models. Taxonomy percentages are relative to valid probes; *None* requires complete canonical trajectory correctness.

Model	Valid	Invalid	State	Transition	Semantic	Horizon	Format	Unknown	None
GPT-5-mini	88.8%	11.2%	10.4%	0.1%	0.0%	49.0%	0.0%	0.0%	40.5%
Kimi K2.5	94.8%	5.2%	10.7%	4.0%	23.2%	56.1%	5.9%	0.0%	0.1%
GLM-5	100.0%	0.0%	84.3%	0.6%	0.0%	14.2%	0.0%	0.0%	1.0%
Qwen 3 Coder 480B A35B	100.0%	0.0%	94.0%	1.1%	0.0%	3.6%	0.0%	0.0%	1.2%
Mistral Large 2512	96.9%	3.1%	11.1%	24.8%	10.2%	49.4%	4.1%	0.0%	0.4%

5 Discussion

5.1 Central Finding

Our central finding is that sequential reliability is multi-faceted rather than reducible to one score. ARCUS-X jointly reports trajectory correctness, prefix persistence, horizon compliance, perturbation profiles, validity, and output overhead. The deterministic taxonomy is applied uniformly across the 13-model publication cohort under strict correctness and canonical-coordinate rules: Table 5 gives the complete validity and failure-signature partition for the five representative models, and Table 15 and Figure 4 extend it to all 13 models.

The important separation is descriptive. In the observed AFS-supported samples, high trajectory accuracy does not imply low output overhead, high horizon compliance does not imply high trajectory accuracy, and two models with similar pooled accuracy can produce very different diagnostic partitions. At the same time, adaptive sampling means these pooled values are not controlled estimates over a common horizon distribution, and the precedence-based taxonomy is not a causal account of internal cognition. ARCUS-X is therefore strongest as an auditable decomposition of observed execution behavior under explicit deterministic rules.

5.2 Limitations

ARCUS-X is synthetic by design: it isolates state-transition execution rather than planning, tool use, memory retrieval, partial observability, changing environments, or irreversible actions, and therefore does not measure general long-horizon agency. Provider/API conditions, response validity, and systems latency may also affect comparisons.

The main empirical limitation is adaptive support. AFS samples horizons nonuniformly from earlier outcomes, so models and tiers can be pooled over different horizon mixtures; completed-probe counts range from roughly 2,200 to 7,600 per model. Cross-model and cross-tier aggregates are therefore descriptive over acquired support, not matched-horizon causal estimates. The post-hoc fracture rule likewise uses three subsequent sampled points with variable spacing, making fracture a property of the sampled condition rather than an intrinsic capacity.

AFS accuracy is conditional on operational validity: unanalyzable responses are excluded from $A(z)$ and reported separately, so long-horizon invalidity can produce insufficient cognitive evidence rather than a low cognitive score. The taxonomy is exhaustive and reproducible but precedence-based and not externally validated; GBI/GE depend on a fixed token allowance and provider/tokenizer accounting; and CRI uses researcher-specified weights. Five probes per batch and five master seeds characterize variation descriptively, not inferentially. Future work should add a common fixed-horizon grid, matched-horizon tier analysis, taxonomy validation, and richer sequential environments.

6 Conclusion

ARCUS-X shows that trajectory correctness, operational validity, output overhead, horizon behavior, and the composition of observed errors can diverge substantially. Across the adaptively sampled runs, models differ not only in pooled accuracy but in where trajectories break, whether requested horizon is preserved, how often output remains analyzable, and which deterministic failure signatures valid errors receive. No single scalar represents all of these behaviors.

The contribution is therefore not a universal ranking or intrinsic "maximum horizon," but an auditable diagnostic record: deterministic tasks and ground truth, raw responses, validity status, per-probe measurements, sampled horizon curves, and an exhaustive post-run partition. AFS directs probes toward threshold-relevant horizons while preserving non-monotonicity, censoring, and operational insufficiency rather than converting them into capacity claims. ARCUS-X measures execution of explicit deterministic state-transition systems under controlled perturbations; it does not establish general agency, naturalistic planning, or tool-use competence.

Generative AI Disclosure Statement

In this work, we used generative AI tools for soundboarding, refining the mathematical metrics and hypotheses, providing feedback on the methodology, and interpreting the results as a cohesive qualitative story. We did not use generative AI tools to generate synthetic datasets, develop proofs, or perform translation; implementation methods were not applicable to this work. We also used generative AI tools to edit LaTeX tables and figures, create and edit code, outline the paper, draft implementation pseudocode for the appendix implementation, make claims accurate to the working codebase, summarize existing literature, and identify gaps in the implementation as it evolved. Generative AI tools were used to polish and improve the clarity and readability of the manuscript at large. We reviewed all AI-assisted work. LLM-generated code was reviewed and audited multiple times, and the authors verified all AI-assisted edits to the paper. We also conducted an independent literature review to complement the generated summaries. We take responsibility for the final content of this work, including text, claims, and artifacts produced with the aid of generative AI.

Reproducibility statement

ARCUS-X is reproducible at the task-generation, evaluation, and artifact levels. Tasks are generated procedurally from explicit master seeds. The same $(\text{seed}, \text{tier}, \text{task index})$ deterministically produces the same grid, initial state, actions, transition rules, label mapping, and ground-truth trajectory. All task randomness is derived from `random.Random`; tier transformations and their ground truth are recomputed deterministically. The reference trajectory is produced by the single transition implementation in `arcus/environment/grid.py`, and `validate_ground_truth()` checks it independently of model output. A stable SHA-256 experiment signature in the code release identifies the generated task inputs; it is not itself a source-revision identifier. The available raw streams retain task parameters and responses, but do not establish the exact prompt text and source revision for every run.

The model prompt contains only the task axioms, transition rules, initial state, grid information, action sequence, question, and required output format. Difficulty metadata, checksums, hidden answers, and final-coordinate shortcuts are excluded, and a SHA-256 prompt digest provides a process-independent prompt identity. The source-controlled parser and metrics pipeline separates trajectory correctness from formatting and execution failures. Step accuracy, exact match, and continuity compare the parsed trajectory with the independently generated path; final-state correctness is reported only as a diagnostic and cannot replace trajectory-level scoring. Specifically, step accuracy is the

number of correct post-initial states divided by $z - 1$ scored transitions; exact match requires identical coordinates and length; and continuity is the longest correct post-initial prefix divided by $z - 1$ (the zero-transition edge case is recorded as zero). The provided initial state is retained for structural comparison but is not counted as a transition. Horizon compliance, parse failures, generation efficiency, fracture depth, latency, and token accounting are retained as additional diagnostics.

Every run records its master-seed list, probe count, gravity and tier settings, horizons, grid sizes, adaptive-search parameters, model identity, compatibility behavior, and source revision in an experiment manifest. Parallel execution assigns each master seed to an isolated runner, so worker ordering does not change benchmark semantics. Interrupted runs can resume from the raw stream: completed probes are identified by master seed, gravity, tier, horizon, probe index, and grid key, then rescored by the same pipeline used for live evaluation. The raw stream preserves model responses, ground truth, task parameters, token counts, latency, and provider adaptations. The separate minitest transcript also records the exact prompts; metrics JSON and benchmark-result JSON provide structured summaries.

The reference environment is the digest-pinned image defined in `Dockerfile`, with dependencies installed from `requirements-lock.txt`; the optional local-inference image similarly pins its CUDA base image and uses `requirements-lock-local.txt`. Source provenance is resolved from `ARCUS_SOURCE_COMMIT` or `git rev-parse HEAD`, and runs fail if neither is available. The public entry point for API evaluation is `arcus.experiments.quickstart`. Validation is run with:

```
python tests/test_arcus_hardening.py
```

An API evaluation is run with:

```
python -m arcus.experiments.quickstart --api-key "$API_KEY" \
  --model <provider>/<model> \
  --n-probes <N> \
  --gravity-levels <g1> <g2> <g3>... \
  --master-seeds <seed-1> <seed-2> <seed-3>...
```

The reference container is validated with:

```
docker build --build-arg \
  SOURCE_COMMIT="$(git rev-parse HEAD)" \
  -t arcus-x:reference .
docker run --rm arcus-x:reference \
  python tests/test_arcus_hardening.py
```

Together, the pinned environment, manifests, raw streams, result matrices, and derived metrics provide an auditable path from source revision and deterministic tasks to the reported results.

REFERENCES

Jing Chen, Yang Sun, Li Zhang, Lin Xu, and Jie Shi. Long-horizon agent trajectory attribution: A unified benchmark and fine-grained annotation framework. *arXiv preprint arXiv:2608.06909*, 2026. URL https://arxiv.org/abs/2608.06909.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL https://arxiv.org/abs/2107.03374.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL https://arxiv.org/abs/2110.14168.

Thomas Grady, Kip Parker, Iliyan Zarov, Henry Course, Chengxi Taylor, and Ross Taylor. Kellybench: A benchmark for long-horizon sequential decision making. *arXiv preprint arXiv:2604.27865*, 2026. URL https://arxiv.org/abs/2604.27865.

Yue Gu et al. Livebench: A challenging, contamination-free llm benchmark. *arXiv preprint arXiv:2406.19314*, 2024. URL https://arxiv.org/abs/2406.19314.

Mads Haahr. RANDOM.ORG: True random number service. https://random.org, 1998. Accessed: 2026-07-27.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020. URL https://arxiv.org/abs/2009.03300.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What's the real context size of your long-context language models? In *Conference on Language Modeling*, 2024. URL https://arxiv.org/abs/2404.06654.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

Aaditya Khanal, Yangyang Tao, and Junxiu Zhou. Beyond pass@1: A reliability science framework for long-horizon llm agents. *arXiv preprint arXiv:2603.29231*, 2026. URL https://arxiv.org/abs/2603.29231.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2308.07201.

Xing Han Lù, Amirhossein Kazemnejad, Nicholas Meade, et al. Agentrewardbench: Evaluating automatic evaluations of web agent trajectories. *arXiv preprint arXiv:2504.08942*, 2025. URL https://arxiv.org/abs/2504.08942.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pp. 13851–13870, 2024. doi: 10.18653/v1/2024.acl-long.747. URL https://aclanthology.org/2024.acl-long.747/.

Akshit Sinha, Arvindh Arun, Shashwat Goel, Steffen Staab, and Jonas Geiping. The illusion of diminishing returns: Measuring long horizon execution in LLMs. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=3lm8lWYxiq.

Jiaxuan Wang, Yulan Hu, Wenjin Yang, Zheng Pan, Xin Li, and Lan-Zhe Guo. Aligning agents via planning: A benchmark for trajectory-level reward modeling. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, 2026. URL https://aclanthology.org/2026.acl-long.1062/.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2410.10813.

Zhaofeng Wu, Linlu Qiu, Alexis Ross, Ekin Akyurek, Boyuan Chen, Bailin Wang, Najoung Kim, Jacob Andreas, and Yoon Kim. Reasoning or reciting? exploring the capabilities and limitations of language models through counterfactual tasks. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 1819–1862, 2024. doi: 10.18653/v1/2024.naacl-long.102. URL https://aclanthology.org/2024.naacl-long.102/.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2307.13854.

A Appendix

A.1 Failure Taxonomy Implementation

This appendix specifies the deterministic classifier used for all failure reports. The implementation is `arcus/analysis/taxonomy.py`: `classify_trajectory` performs the top-level validity, correctness, and formatting decisions, and `_attribute_mechanism` assigns the primary mechanism for a valid incorrect trajectory. `classify_from_result` is the runner and post-run-analyzer entry point. The serialized `ProbeClassification` stores the validity flag, error code, mechanism, first failure step, secondary pattern flags, trajectory-correctness flag, output-compliance status, and formatting subcode.

A.1.1 Output-validity partition

The classifier first determines whether a response can provide cognitive evidence. An exception marker yields `E100_EXCEPTION_TOKEN`; an empty response yields `E101_EMPTY_OUTPUT`; an unparseable coordinate sequence yields `E102_PARSE_FAILURE`; and uncategorized invalid responses yield `E104_UNKNOWN_INVALID`. A classifier exception is recorded as a provider error (`E103_PROVIDER_ERROR`). Invalid outputs are marked *unclassifiable* and excluded from the cognitive-failure denominator. The post-run analyzer verifies $N_{\text{valid}} + N_{\text{invalid}} = N_{\text{completed}}$.

A.1.2 Primary mechanisms and precedence

For a parseable response, the primary taxonomy applies these decisions in order:

1. A full-length modulo-equivalent path containing noncanonical coordinates is an output-format failure, not success, unless an out-of-range step follows the unwrapped transition rule; that boundary violation retains transition-failure precedence.
2. A complete, literal match to the full canonical reference path is *None*. Verbose or Markdown-wrapped correct responses remain correct; output compliance is stored separately.
3. An initial-state mismatch is a semantic-interpretation failure.
4. A wraparound-only boundary error is a transition-rule failure.
5. If all overlapping transitions are otherwise correct, a length mismatch is a horizon failure.
6. After a correct prefix, persistent offset or increasing drift is a state-tracking failure. Otherwise, a majority of structured transition errors is a transition-rule failure; immediate increasing divergence is a horizon failure; remaining valid early-divergence cases use the conservative state-tracking fallback.
7. For a valid parsed response with zero aligned matches and a length mismatch, output-format failure overrides the preceding mechanism.

Responses that fail operational parsing are invalid and excluded from this trajectory-level taxonomy. The classifier also records nonexclusive secondary patterns and temporal dynamics. Reports normalize implementation labels to *State Tracking Failure*, *Transition Rule Failure*, *Semantic Interpretation Failure*, *Horizon Collapse*, and *Formatting Failure*. A valid trajectory that cannot be assigned to one of these categories is *Unknown / Unmapped*; it is never counted as *None*. The analysis also records fallback assignments separately, and Analysis B reports their reclassification as a sensitivity policy. These deterministic labels describe observable output under this precedence, not latent cognitive causes.

A.2 Operational and Temporal Failure Analysis

The primary ARCUS-X metrics characterize trajectory correctness, but two additional distinctions are useful when interpreting long-horizon behavior. First, a model must produce an analyzable response before trajectory correctness can be measured. Second, aggregate step accuracy does not identify *where* within a valid trajectory errors emerge. We therefore separate operational validity from conditional trajectory correctness and perform a post-hoc temporal localization analysis over valid trajectories.

A.2.1 Operational versus Cognitive Reliability

ARCUS-X separates *operational reliability* from *cognitive reliability*. Operational reliability concerns whether a completed evaluation produces an output that can be parsed and scored under the benchmark protocol. Cognitive reliability, as used here, refers only to correctness of the observable state-transition execution conditional on such an analyzable response; it is not a claim about a model's internal cognitive process.

For horizon z, let V_z denote the set of probes yielding valid, analyzable trajectories, and let $a_i \in [0, 1]$ denote the ordinary step accuracy of valid probe i. Horizon-level cognitive accuracy is

$$A(z) = \frac{1}{|V_z|} \sum_{i \in V_z} a_i. \tag{1}$$

Operationally invalid probes are excluded from this denominator rather than assigned a cognitive accuracy of zero. They remain explicitly represented in the operational validity statistics and error-code accounting. ARCUS-X records E100 for exception-token outputs, E101 for empty outputs, E102 for parse failures, E103 for provider or evaluation exceptions, and E104 for other invalid responses. The accounting invariant is

$$N_{\text{valid}} + N_{\text{invalid}} = N_{\text{completed}}. \tag{2}$$

If $|V_z| = 0$, then $A(z)$ is undefined. ARCUS-X records the corresponding horizon as operationally insufficient evidence rather than setting $A(z) = 0$. The same rule is used by Adaptive Fracture Search: an all-invalid batch does not constitute evidence that the model crossed the cognitive fracture threshold.

This separation prevents two observably different failure modes from being collapsed into one statistic. A model may continue to return well-formed trajectories while increasingly misexecuting the supplied transition rules. In that case, the benchmark observes degradation in sequential execution. Alternatively, a model may increasingly return empty, malformed, or otherwise unanalyzable outputs. This is an end-to-end reliability failure, but no trajectory exists from which a cognitive step-accuracy value can be measured. Assigning such responses a value of zero would therefore mix response availability with conditional execution correctness.

Operational failures are not discarded. They are reported on their own axis and remain part of the overall reliability profile. The separation only prevents the absence of analyzable evidence from being interpreted as an observed incorrect trajectory.

A.2.2 Temporal Localization within Valid Trajectories

Conditional on operational validity, we next ask where within a trajectory correctness deteriorates. Overall step accuracy can conceal substantially different execution profiles. Two trajectories may have similar aggregate accuracy even if one fails immediately while another executes correctly for most of the requested horizon before degrading near the end.

For each valid probe at horizon z, the z requested transition indices are partitioned into three contiguous, as-even-as-possible regions: *early*, *middle*, and *late*. When z is not divisible by three, segment lengths differ by at most one transition. As in the primary step-accuracy metric, the provided initial state is excluded.

Table 6: Within-trajectory step accuracy at two common requested horizons for the 13-model publication cohort. Equal horizon does not imply identical tasks; this table is descriptive and is not task-paired unless exact task hashes match. Early, middle, and late correspond to contiguous thirds of the scored transition sequence. Values are conditional on valid, analyzable trajectories. The table is a positional diagnostic rather than a model ranking.

	$z = 20$			$z = 30$		
Model	Early	Middle	Late	Early	Middle	Late
GPT-5-mini	99.8%	99.9%	100.0%	100.0%	99.8%	99.4%
Kimi K2.5	87.9%	86.5%	83.3%	69.8%	67.0%	66.2%
GLM-5	95.9%	84.3%	61.0%	93.6%	67.3%	37.3%
Qwen3-Coder-480B-A35B	93.5%	61.7%	30.2%	83.5%	41.7%	17.4%
Qwen3-235B-A22B-2507	49.2%	49.4%	47.6%	50.4%	47.0%	44.1%
Mistral Large 2512	56.4%	56.3%	56.1%	77.7%	76.9%	76.6%
Mistral Small 2603	74.1%	47.2%	41.5%	67.9%	23.1%	15.4%
Devstral 2 123B	50.6%	51.2%	51.5%	55.4%	55.8%	56.5%
Ministral 14B 2512	88.1%	82.2%	77.8%	85.4%	74.2%	68.8%
Ministral 8B 2512	58.4%	55.0%	51.3%	65.9%	55.7%	47.7%
Ministral 3B 2512	75.6%	59.7%	47.0%	69.4%	43.2%	28.6%
Nemotron Super 3 120B	89.3%	44.1%	19.4%	80.3%	22.7%	4.7%
MAI-Thinking-1	100.0%	100.0%	100.0%	99.4%	98.8%	98.3%

Let $T_r(z)$ denote the transition indices assigned to temporal region $r \in \{\text{early}, \text{middle}, \text{late}\}$. Using the same per-transition correctness indicators as the primary metric, regional accuracy is

$$A_r(z) = \frac{\sum_{i \in V_z} \sum_{t \in T_r(z)} \mathbb{1}[\hat{s}_{i,t} = s_{i,t}]}{\sum_{i \in V_z} |T_r(z)|}. \tag{3}$$

The temporal partition is purely positional. It does not assign a causal failure type to an individual transition. In particular, the ARCUS-X error taxonomy remains a *trajectory-level* diagnostic and is not duplicated across temporal thirds. Temporal localization answers *where* observed correctness is lost; the taxonomy separately describes the deterministic failure signature assigned to the complete trajectory.

Matched-horizon comparison. Because AFS produces nonuniform horizon support, pooling temporal statistics across all acquired horizons would inherit the same support differences as the headline aggregate metrics. We therefore use $z = 20$ and $z = 30$ for the cross-model comparison below. Both requested horizons are represented across the reported cohort, permitting a descriptive comparison at common horizon values. Equal horizon does not imply identical task conditions: paired comparisons are restricted to observations with identical task hashes and are reported separately in the correction package.

A within-trajectory accuracy gradient is visible in a majority of the observed profiles. At $z = 20$, 9 of the 13 models have strictly decreasing accuracy from the early to middle to late region. At $z = 30$, this pattern occurs for 12 of 13 models. The magnitude, however, differs substantially across models.

Several models preserve high initial-step accuracy while degrading sharply later in the same trajectory. At $z = 30$, the early, middle, and late values are 80.3%, 22.7%, 4.7% for Nemotron Super 3, 83.5%, 41.7%, 17.4% for Qwen3-Coder, and 93.6%, 67.3%, 37.3% for GLM-5. These profiles

would be substantially compressed by a single trajectory-level mean: the model can execute a large fraction of early transitions correctly even when the terminal portion is already unreliable.

The pattern is not universal. At the same $z = 30$ horizon, Kimi K2.5 has 69.8%, 67.0%, 66.2%, Mistral Large has 77.7%, 76.9%, 76.6%, and MAI-Thinking-1 has 99.4%, 98.8%, 98.3%. Devstral 2 is approximately flat with 55.4%, 55.8%, 56.5%. We therefore do not interpret late-region degradation as a universal monotonic law of sequential execution.

Development at longer horizons. The onset of temporal degradation is itself horizon dependent. GPT-5-mini, for example, remains nearly uniform through moderate horizons: at $z = 80$ its early, middle, and late accuracies are 99.7%, 98.6%, 97.6%, respectively. The separation becomes substantially larger among the longest sampled trajectories. At $z = 115$, the corresponding values are 99.7%, 96.5%, 88.4%; at $z = 133$, they are 99.3%, 91.5%, 78.1%; and at $z = 149$, they are 98.2%, 84.5%, 67.1%. Thus, aggregate long-horizon degradation can arise even when the beginning of the requested trajectory remains almost perfectly executed.

Other models exhibit the same qualitative separation at much shorter horizons. GLM-5 has early, middle, and late values of 95.9%, 84.3%, 61.0% at $z = 20$, and 79.8%, 27.5%, 8.5% at $z = 60$. Qwen3-Coder and Nemotron Super 3 show sharper early-to-late separation by $z = 30$. The horizon at which a temporal gradient becomes visible is therefore model dependent and cannot be recovered from a single pooled accuracy statistic.

Adjacent horizons can also recover or fluctuate. We consequently do not fit a monotonic temporal-decay model or interpolate unobserved horizons. As elsewhere in ARCUS-X, the reported values describe only sampled, observed support.

Relation to the trajectory-level taxonomy. Temporal localization and the deterministic error taxonomy provide complementary views of the same observed output. Models with particularly large early-to-late separation, including GLM-5, Qwen3-Coder, and Nemotron Super 3, are also dominated by State Tracking Failure in the trajectory-level taxonomy. In contrast, models with flatter temporal profiles can exhibit different error compositions, including larger semantic or transition-related components.

This correspondence is descriptive rather than causal. A steep temporal gradient is compatible with accumulated state drift, but the thirds analysis does not establish that an early error caused a later error, nor does it identify an internal mechanism. Likewise, a trajectory-level taxonomy label is not retrospectively assigned to each region. The two diagnostics answer different questions: temporal localization identifies where correctness is lost within the observed sequence, while the taxonomy characterizes the failure signature of the trajectory as a whole.

Interpretation and limitations. The temporal analysis is conditional on operational validity. Horizons with no valid trajectories remain N/A and are not imputed as zero. Consequently, a model with strong conditional early/middle/late accuracy but low output validity should not be interpreted as strongly reliable end-to-end; its operational behavior must be considered separately.

The three-region partition is intentionally coarse. It exposes positional structure without introducing another fitted failure detector, but it does not identify an exact failure transition. In addition, because ARCUS-X trajectories are sequential, later incorrect states may be consistent with accumulated earlier drift, independent mistakes, or recovery after a previous mismatch. The regional statistics do not distinguish among these possibilities.

Finally, AFS produces different horizon support across models and conditions. The matched $z = 20$ and $z = 30$ comparison above holds requested horizon fixed for the cross-model table, while longer-horizon examples are interpreted only within their observed model-specific support. These results therefore strengthen the diagnostic decomposition of ARCUS-X—showing that long-horizon failure can be strongly localized within an otherwise valid trajectory—without converting the observed temporal profile into an intrinsic model capacity or causal account of failure.

A.3 AFS IMPLEMENTATION

This appendix documents the implementation used by the benchmark runner. AFS is not a separate scoring pipeline or model: it reuses the benchmark's task generation, trajectory extraction, scoring, taxonomy, and operational-validity checks. The active search is implemented in `BenchmarkRunner`; `FracturePointFinder` is used only for post-hoc analysis of an already sampled accuracy curve.

A.3.1 EVALUATION UNIT AND SCORING

For each gravity-tier condition and candidate horizon z, the runner generates a batch of seeded probes. The generator computes the ground-truth trajectory from the deterministic transition system, while the evaluator returns a model trajectory. For every response, the shared benchmark path extracts the predicted trajectory and computes step accuracy by comparing it with ground truth. The runner then applies the cognitive-validity gate before aggregating evidence. Thus, live AFS and resume reconstruction score all z action transitions; the reported trajectory tables score the requested z coordinates.

Let R_z denote the responses in a batch and let $V_z \subseteq R_z$ be the responses that pass the validity gate. The live search statistic is

$$A(z) = \begin{cases} \frac{1}{|V_z|} \sum_{i \in V_z} a_i, & |V_z| > 0, \\ \text{None}, & |V_z| = 0, \end{cases}$$

where $a_i \in [0, 1]$ is step accuracy. Provider and API failures, error terminations, invalid classifications, empty output, parse failure, and other operationally invalid responses are excluded from $A(z)$ and remain available for failure reporting. In particular, invalid responses are not converted to zero accuracy.

A.3.2 LIVE SEARCH CONTROLLER

The accuracy floor is $\tau = 0.5$. For an adaptive run, the search starts with $low = 1$ and $high = \min(40, H)$, where H is the configured hard ceiling (150 by default). Although ARCUS-X's search window can be adjusted, we use $z = 150$ for reproducibility. At each iteration, it evaluates $z_{\text{mid}} = \left\lfloor \frac{low+high}{2} \right\rfloor$. If $A(z_{\text{mid}})$ is `None` or is at least τ, the search treats the result as not providing cognitive-fracture evidence and advances deeper by setting $low = z_{\text{mid}} + 1$. A `None` batch is separately reported as operational insufficiency; it is not evidence that the model has cognitively survived that horizon. When the evaluated midpoint reaches the current upper bound and still passes, the upper bound may be doubled, up to H. If $A(z_{\text{mid}}) < \tau$, the midpoint is recorded as the current fracture candidate and the search retreats by setting $high = z_{\text{mid}} - 1$. The loop terminates when $low > high$ or the iteration limit is reached. If no below-floor midpoint is found and the search passes beyond H, the reported fallback is H; otherwise the recorded candidate is retained (or remains zero when the search did not establish one). Consequently, a ceiling result with operational insufficiency must not be interpreted as a cognitive capacity estimate.

The non-adaptive path evaluates the configured target horizon directly rather than running the bisection loop. Equality at τ is passing because the fracture comparison is strict ($A(z) < \tau$). Consequently, the returned search value is a directly evaluated candidate found by this controller, but it is not necessarily the mathematically first failing horizon when accuracy is noisy or non-monotonic.

A.3.3 POST-HOC FRACTURE ANALYSIS

The analysis-layer `FracturePointFinder` scans a sampled accuracy curve. When a point is below τ, it examines the next `look_ahead_step` points (three by default). The post-run tables use a two-point window. If any point in the selected window returns to at least τ, the candidate dip is suppressed; otherwise the point is reported as a structural fracture. This is a forward-window recovery check, not a requirement for three consecutive failures, not three additional probes, and not a decision rule used by the live bisection loop. It is therefore important to distinguish the search candidate from a post-hoc structural-yield report.

A.3.4 CACHING, RESUME, AND REPRODUCIBILITY

Probe identity for reuse is keyed by seed, gravity, tier, horizon, probe index, and grid key. Resume logic deduplicates matching observations and validates model metadata and, for offline resume, the experiment hash; mismatches raise an error rather than silently mixing runs. The seeded generator and integer search arithmetic are deterministic conditional on identical model outputs, provider behavior, and raw-stream state. Seed alone does not guarantee identical AFS results because model responses, timeouts, truncation, rate limits, and resume ordering can differ. The controller is adaptive only in the narrow sense that prior batch accuracies determine interval updates; it is a deterministic midpoint search over the horizon axis, not a stochastic or semantic heuristic.

A.4 ADAPTIVE HORIZON SEARCH AND SAMPLING

ARCUS-X does not evaluate every possible execution horizon uniformly. Instead, the Adaptive Fracture Search (AFS) procedure allocates evaluations adaptively to localize the interval in which sequential reliability falls below the fracture floor, without exhaustively evaluating every horizon. This makes the search a bounded, bisection-style allocation strategy rather than an exhaustive scan across all integer horizons.

Figure 2 shows the pooled observed per-horizon step accuracy and probe density for **Ministral 14B 2512** (run identifier: `ministral-14b-2512`), aggregated across its sampled conditions. Only horizons observed in the run are plotted. Point color encodes probe density, so the plot displays both the measured accuracy and where AFS allocated probes. It is a sampling diagnostic, not a tier-specific fracture curve.

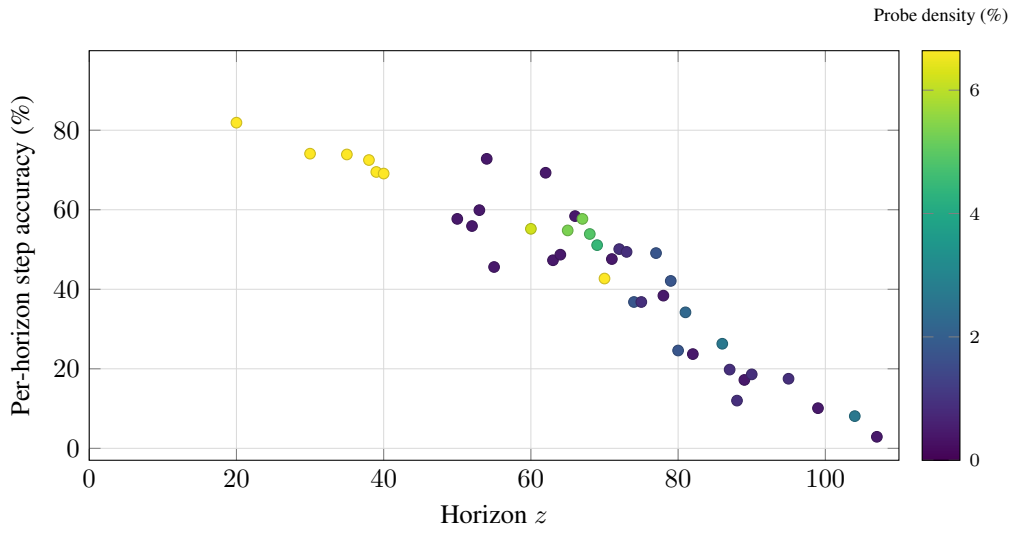


Figure 2: Pooled observed horizon accuracy and probe density for Ministral 14B 2512 across sampled conditions, excluding calibration probes. Each point is an observed horizon; color encodes probe density (percent of probes).

The density varies across sampled horizons. This descriptive plot does not establish the cause of the observed accuracy pattern.

A.5 CRI

The optional Composite Robustness Index (CRI) provides a secondary summary across evaluation axes: $\text{CRI} = 0.45\,\text{TF} + 0.30\,\text{HR} + 0.20\,\text{SR} + 0.05\,\text{GE}$.

Its components are:

1. **Trajectory Fidelity** $\text{TF} = 0.5A + 0.3C + 0.2E$, where A is step accuracy, C is continuity, and E is exact-match rate.

2. **Horizon Robustness**, defined as the normalized trapezoidal area under the accuracy-horizon curve:

$$\mathrm{HR} = \frac{\sum_i \frac{A_i + A_{i+1}}{2}(z_{i+1} - z_i)}{z_{\max} - z_{\min}}.$$

3. **Semantic Robustness** $\mathrm{SR} = 0.5R_1 + 0.25R_2 + 0.25R_3$, where $R_i = \min(1, A_i/A_0)$ is the ratio of Tier-i accuracy to Tier-0 accuracy, clamped to $[0, 1]$. If Tier-0 accuracy is zero or unavailable, SR is the arithmetic mean of the available tier accuracies. Tier 1 receives the larger weight because it is a label-only perturbation and leaves the transition function unchanged.

A.6 Seed-Level Variability

Table 7 reports the population mean and standard deviation of the secondary CRI across the five master seeds for each run. These values characterize seed sensitivity; they are not confidence intervals and should not be used to claim statistically significant between-model differences.

Table 7: CRI across five master seeds (population mean $\pm$ standard deviation). Asterisks indicate response validity below 75%.

Model	CRI
qwen.qwen3-235b-a22b-2507*	0.250 ± 0.027
Kimi K2.5	0.567 ± 0.032
Qwen 3 Coder 480B A35B	0.517 ± 0.021
mistral-small-2603	0.505 ± 0.023
Mistral Large 2512	0.538 ± 0.016
mistral.devstral-2-123b*	0.500 ± 0.026
ministral-14b-2512	0.556 ± 0.065
ministral-8b-2512	0.501 ± 0.041
ministral-3b-2512	0.479 ± 0.047
nvidia.nemotron-super-3-120b	0.519 ± 0.032
GLM-5	0.590 ± 0.032
GPT-5-mini	0.814 ± 0.034
MAI-Thinking-1*	0.480 ± 0.049

A.7 Full Model Results

Table 8: Complete overall ARCUS-X results for all 13 evaluated models. Asterisks indicate response validity below 75%.

Model	Step Acc.	Exact Match	Continuity	Horizon Compliance
qwen.qwen3-235b-a22b-2507*	14.1%	0.3%	11.9%	17.3%
Kimi K2.5	59.2%	0.1%	53.9%	77.0%
Qwen 3 Coder 480B A35B	49.7%	1.2%	43.1%	98.3%
mistral-small-2603	50.3%	0.6%	46.8%	80.1%
Mistral Large 2512	56.0%	0.4%	53.0%	45.2%
mistral.devstral-2-123b*	44.7%	0.1%	43.3%	42.0%
ministral-14b-2512	55.5%	5.9%	51.6%	74.5%
ministral-8b-2512	47.7%	0.9%	45.0%	64.2%
ministral-3b-2512	43.7%	8.4%	40.2%	72.7%
nvidia.nemotron-super-3-120b	47.7%	0.6%	42.4%	96.8%
GLM-5	56.2%	1.0%	51.3%	98.4%
GPT-5-mini	85.3%	36.0%	85.0%	87.7%
MAI-Thinking-1*	40.0%	6.4%	40.0%	40.1%

A.7.1 HORIZON-DEPENDENT DEGRADATION AND FRACTURE

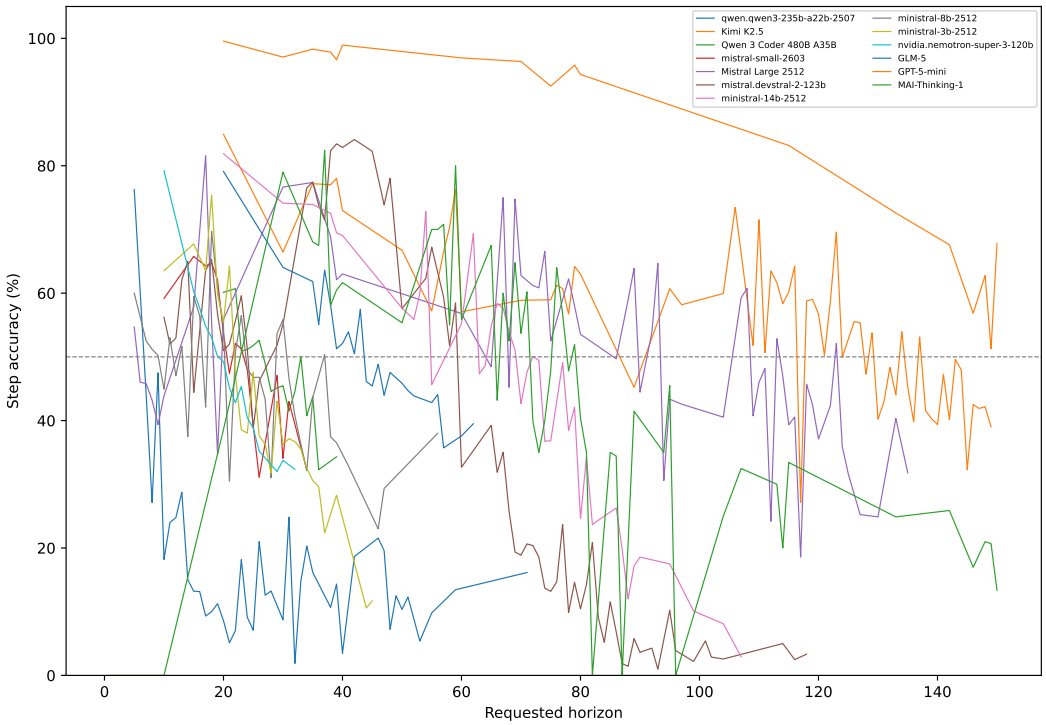


Figure 3: Observed per-horizon accuracy for every evaluated model, pooled over all recorded non-calibration probes. Curves join only horizons sampled by the adaptive procedure.

Table 9: Post-hoc structural fracture depth by perturbation tier for all 13 evaluated models. Accuracy is pooled over all recorded non-calibration probes, including invalid responses. Values are the smallest sampled horizon below the 50% step-accuracy floor that is not followed by a recovery within the next two sampled horizons; 'N/A' means that no such sampled point was found. Asterisks indicate response validity below 75%.

Model	Tier 0	Tier 1	Tier 2	Tier 3
qwen.qwen3-235b-a22b-2507*	13	7	10	5
Kimi K2.5	145	N/A	130	55
Qwen 3 Coder 480B A35B	34	30	29	23
mistral-small-2603	21	34	21	23
Mistral Large 2512	9	5	94	6
mistral.devstral-2-123b*	10	58	24	13
ministral-14b-2512	69	82	62	40
ministral-8b-2512	21	26	8	12
ministral-3b-2512	22	26	25	23
nvidia.nemotron-super-3-120b	24	20	24	16
GLM-5	57	44	56	30
GPT-5-mini	N/A	N/A	N/A	N/A
MAI-Thinking-1*	1	1	1	1

The table reports the structural-yield analysis of the sampled curves, not an exhaustive scan of every integer horizon. The live AFS controller starts with an upper bound of $\min(40, H)$ and doubles that bound only after a passing (or operationally insufficient) midpoint reaches the current upper edge; expansion stops at the absolute hard ceiling $H = 150$. We call the search window *saturated* when no below-floor midpoint has been found and the lower search bound advances beyond H; the recorded live-search value is then the right-censored ceiling, $z = 150$. This ceiling fallback is not evidence of 150-step cognitive survival, especially when batches are operationally insufficient. Conversely,

the 'N/A' entries above mean that the post-hoc two-point recovery rule found no persistent sampled fracture; they are not synonymous with a saturated live search.

Table 10: Per-horizon accuracy fluctuation over observed horizons. Std. dev. is the population standard deviation of each model's per-horizon accuracies rounded to one decimal place. Asterisks indicate response validity below 75%.

Model	Observations (N)	Mean Accuracy	Std. Dev.
qwen.qwen3-235b-a22b-2507*	43	16.90%	12.98%
Kimi K2.5	64	55.75%	12.09%
Qwen 3 Coder 480B A35B	17	46.98%	7.41%
mistral-small-2603	16	49.71%	11.26%
Mistral Large 2512	61	50.24%	13.88%
mistral.devstral-2-123b*	73	36.84%	27.47%
ministral-14b-2512	41	44.81%	20.61%
ministral-8b-2512	38	46.01%	10.15%
ministral-3b-2512	26	41.56%	16.30%
nvidia.nemotron-super-3-120b	16	46.88%	12.06%
GLM-5	24	50.55%	9.67%
GPT-5-mini	19	85.33%	15.95%
MAI-Thinking-1*	55	41.61%	23.06%

Table 11: Pearson correlation between horizon-accuracy fluctuation and trajectory-level metrics across all models.

Metric	Pearson r with Fluctuation
Horizon Compliance	-0.499
Exact Match	0.172
Step Accuracy	-0.060
Continuity	0.023

With only 13 model observations, these Pearson correlations are descriptive and exploratory rather than evidence of general population relationships. The fracture-depth correlation is omitted because fracture observations are right-censored at the search ceiling and absent for more than one model. Within this evaluation, greater fluctuation co-occurs with lower horizon compliance, while the relationships with step accuracy and continuity are near zero and exact match is weakly positive. These associations should not be interpreted as general claims about predictability beyond this sample.

A.7.2 Semantic Robustness

Table 12 reports per-tier step accuracy for all 13 evaluated models, since the benchmark runner evaluates all four tiers at every configured gravity value (Section 3.4). Table 13 reports the corresponding CRI decomposition, including the Semantic Robustness (SR) component (Appendix A.5).

Table 12: Complete per-tier step accuracy for all 13 evaluated models. Asterisks indicate response validity below 75%.

Model	Tier 0	Tier 1	Tier 2	Tier 3
qwen.qwen3-235b-a22b-2507*	17.2%	15.1%	19.0%	5.2%
Kimi K2.5	63.1%	75.8%	65.2%	32.9%
Qwen 3 Coder 480B A35B	56.9%	49.7%	53.7%	38.4%
mistral-small-2603	47.7%	63.3%	46.6%	43.8%
Mistral Large 2512	56.1%	48.6%	58.6%	60.5%
mistral.devstral-2-123b*	44.7%	52.3%	44.4%	37.3%
ministral-14b-2512	56.2%	60.9%	55.8%	49.0%
ministral-8b-2512	48.4%	47.0%	45.3%	50.3%
ministral-3b-2512	43.3%	44.3%	42.7%	44.3%
nvidia.nemotron-super-3-120b	56.8%	45.4%	50.7%	37.9%
GLM-5	64.5%	58.7%	60.3%	41.3%
GPT-5-mini	88.0%	83.7%	87.5%	82.2%
MAI-Thinking-1*	45.0%	29.1%	46.7%	39.2%

Table 13: CRI and its decomposition for all 13 evaluated models: Trajectory Fidelity (TF), Horizon Robustness (HR), Semantic Robustness (SR), and Generation Efficiency (GE), pooled across probes (Appendix A.5). Asterisks indicate response validity below 75%.

Model	TF	HR	SR	GE	CRI
qwen.qwen3-235b-a22b-2507*	0.107	0.073	0.764	0.823	0.264
Kimi K2.5	0.458	0.498	0.880	0.559	0.559
Qwen 3 Coder 480B A35B	0.380	0.404	0.841	0.994	0.510
mistral-small-2603	0.393	0.296	0.973	0.603	0.491
Mistral Large 2512	0.440	0.502	0.933	0.238	0.547
mistral.devstral-2-123b*	0.353	0.306	0.957	0.551	0.470
ministral-14b-2512	0.444	0.381	0.967	0.361	0.525
ministral-8b-2512	0.375	0.364	0.969	0.266	0.485
ministral-3b-2512	0.356	0.343	0.997	0.214	0.473
nvidia.nemotron-super-3-120b	0.367	0.547	0.790	0.999	0.537
GLM-5	0.437	0.535	0.848	1.000	0.577
GPT-5-mini	0.754	0.836	0.958	0.256	0.794
MAI-Thinking-1*	0.333	0.436	0.790	0.700	0.474

The CRI values in Table 13 are pooled single-run summaries and differ slightly from the per-seed population means reported in Appendix A.6, which average five independently computed CRI values per model. The two summaries are not interchangeable: the pooled CRI aggregates all probes as a single sample, while the seed-level mean averages five separately computed run-level statistics.

A.8 Generation Efficiency and Failure Modes

Table 14: Generation efficiency measurements. Asterisks indicate response validity below 75%.

Model	Step Accuracy	GBI	Generation Efficiency	Correct Steps/sec
qwen.qwen3-235b-a22b-2507*	14.1%	1.44	82.3%	0.283
Kimi K2.5	59.2%	1.12	55.9%	2.406
Qwen 3 Coder 480B A35B	49.7%	0.02	99.4%	6.786
mistral-small-2603	50.3%	1.58	60.3%	3.083
Mistral Large 2512	56.0%	4.49	23.8%	1.127
mistral.devstral-2-123b*	44.7%	1.91	55.1%	1.131
ministral-14b-2512	55.5%	2.39	36.1%	1.411
ministral-8b-2512	47.7%	3.33	26.6%	0.930
ministral-3b-2512	43.7%	4.91	21.4%	1.543
nvidia.nemotron-super-3-120b	47.7%	0.03	99.9%	4.278
GLM-5	56.2%	0.00	100.0%	4.626
GPT-5-mini	85.3%	4.25	25.6%	1.922
MAI-Thinking-1*	40.0%	1.41	70.0%	0.694

Table 15 reports the complete validity and taxonomy partition for all 13 evaluated models, extending the five-model view in Table 5. Figure 4 visualizes the five primary failure categories; *None*, Unknown / Unmapped, and invalid responses remain visible in the table rather than the heatmap.

Table 15: Post-run validity and taxonomy partition for the 13-model publication cohort. Taxonomy percentages are relative to valid probes; invalid responses are reported separately.

Model	Valid	Invalid	State	Transition	Semantic	Horizon	Format	Unknown	None
qwen.qwen3-235b-a22b-2507*	26.0%	74.0%	8.2%	2.0%	37.2%	40.2%	11.0%	0.0%	1.3%
Kimi K2.5	94.8%	5.2%	10.7%	4.0%	23.2%	56.1%	5.9%	0.0%	0.1%
Qwen 3 Coder 480B A35B	100.0%	0.0%	94.0%	1.1%	0.0%	3.6%	0.0%	0.0%	1.2%
mistral-small-2603	100.0%	0.0%	47.8%	17.6%	0.3%	33.5%	0.0%	0.0%	0.6%
Mistral Large 2512	96.9%	3.1%	11.1%	24.8%	10.2%	49.4%	4.1%	0.0%	0.4%
mistral.devstral-2-123b*	72.7%	27.3%	23.7%	23.9%	0.7%	50.9%	0.8%	0.0%	0.1%
ministral-14b-2512	91.2%	8.8%	50.6%	11.1%	1.3%	29.9%	0.7%	0.0%	6.4%
ministral-8b-2512	99.7%	0.3%	21.2%	40.4%	0.3%	36.8%	0.4%	0.0%	0.9%
ministral-3b-2512	82.0%	18.0%	56.9%	4.6%	6.4%	16.2%	5.7%	0.0%	10.2%
nvidia.nemotron-super-3-120b	99.9%	0.1%	92.6%	1.1%	0.0%	5.4%	0.2%	0.0%	0.6%
GLM-5	100.0%	0.0%	84.3%	0.6%	0.0%	14.2%	0.0%	0.0%	1.0%
GPT-5-mini	88.8%	11.2%	10.4%	0.1%	0.0%	49.0%	0.0%	0.0%	40.5%
MAI-Thinking-1*	40.8%	59.2%	3.0%	0.0%	0.0%	81.2%	0.0%	0.0%	15.8%

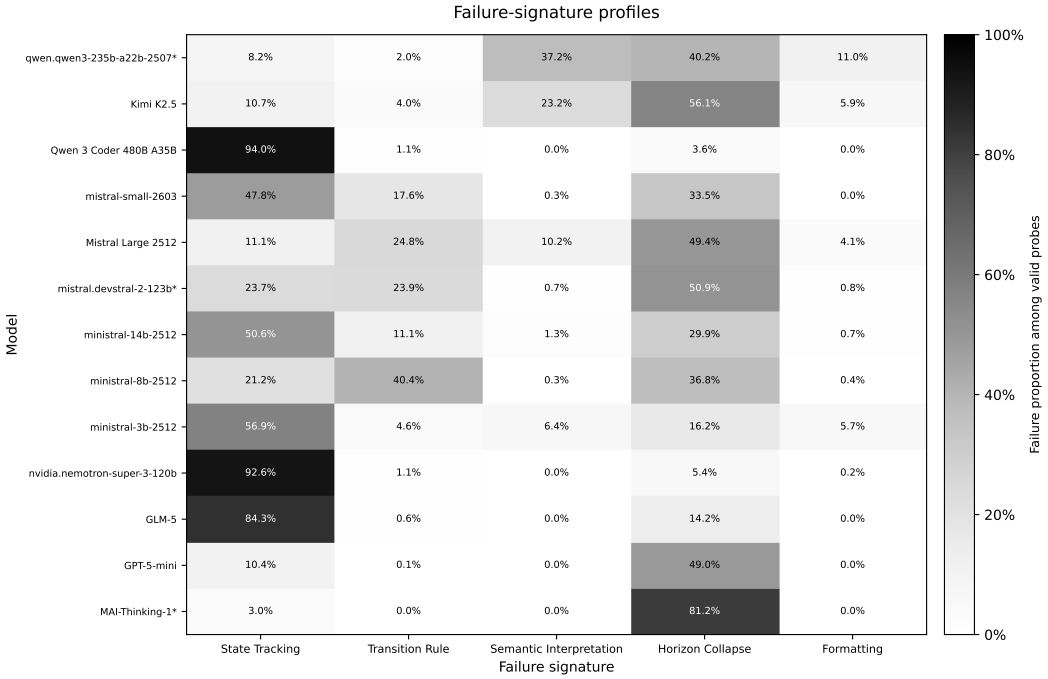


Figure 4: Failure-signature profiles across the 13-model publication cohort, expressed as a percentage of valid probes. Asterisks in the row labels identify models with response validity below 75%.

For models with low response validity, the taxonomy columns describe only the analyzable subset and may therefore be less representative of all completed probes.

A.9 Taxonomy Policy Sensitivity (Analysis B)

Analysis A is the primary analysis and uses the classifier with strict trajectory correctness, canonical coordinates, and the primary precedence. Analysis B changes one policy at a time; it does not replace Analysis A. The table reports, for each manuscript model, the percentage of valid non-calibration observations whose primary label changes under each sensitivity policy. Conservative reclassification maps fallback, unknown, and any non-exact *None* to Unclassified; fallback-only reclassification maps only state-tracking fallback cases; horizon-first and transition-first reorder the corresponding detected indicator; modulo sensitivity treats full-length modulo-equivalent noncanonical paths as correct. Probe-level labels, including calibration and the additional Minimax M2.5 log, are in `analysis_B_probe_level.csv`.

B Minitest transcripts

These are the recorded prompts, model responses, and reference trajectories for Ministral-14B-2512 at $z = 50$, seed 3157, and gravity 0.5 across all four tiers. The prompts are reproduced as sent.

```
=== TIER 0 ===

[EXACT SENT PROMPT]:
Rules:
  - The world is a grid with width 5 and height 8. Coordinates are (x, y) with x in [0, 4] and
  ↪  y in [0, 7].
  - The grid is toroidal: movement off one edge wraps to the opposite edge.
  - The agent starts at position (2, 7).
  - Transition rules:
  -   - When the action is 'south', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'south', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'north', the agent moves by (dx=0, dy=1).
```

Table 16: Share of valid probes reassigned relative to Analysis A under alternative taxonomy policies.

Model	Conservative	Fallback	Horizon-first	Transition-first	Modulo
qwen.qwen3-235b-a22b-2507*	0.0%	0.0%	54.1%	7.7%	0.0%
Kimi K2.5	0.0%	0.0%	42.6%	10.5%	0.0%
Qwen 3 Coder 480B A35B	0.0%	0.0%	28.6%	92.3%	0.0%
mistral-small-2603	0.0%	0.0%	30.7%	47.7%	0.0%
Mistral Large 2512	0.0%	0.0%	49.3%	11.0%	0.0%
mistral.devstral-2-123b*	0.0%	0.0%	46.0%	23.3%	0.0%
ministral-14b-2512	0.0%	0.0%	39.7%	50.3%	0.0%
ministral-8b-2512	0.0%	0.0%	59.3%	20.5%	0.0%
ministral-3b-2512	0.1%	0.1%	34.6%	55.5%	0.2%
nvidia.nemotron-super-3-120b	0.0%	0.0%	45.8%	90.8%	0.0%
GLM-5	0.0%	0.0%	39.1%	80.8%	0.0%
GPT-5-mini	0.0%	0.0%	2.8%	10.4%	0.0%
MAI-Thinking-1*	0.0%	0.0%	2.5%	2.9%	0.0%

```
  -   - When the action is 'north', the agent moves by (dx=0, dy=1).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'south', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'north', the agent moves by (dx=0, dy=1).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'north', the agent moves by (dx=0, dy=1).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'north', the agent moves by (dx=0, dy=1).
  -   - When the action is 'north', the agent moves by (dx=0, dy=1).
  -   - When the action is 'south', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'north', the agent moves by (dx=0, dy=1).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'south', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'south', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'south', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'north', the agent moves by (dx=0, dy=1).
  -   - When the action is 'south', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'south', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'north', the agent moves by (dx=0, dy=1).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'north', the agent moves by (dx=0, dy=1).
  -   - When the action is 'north', the agent moves by (dx=0, dy=1).
  -   - When the action is 'south', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'north', the agent moves by (dx=0, dy=1).
  -   - When the action is 'north', the agent moves by (dx=0, dy=1).
  -   - When the action is 'north', the agent moves by (dx=0, dy=1).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'north', the agent moves by (dx=0, dy=1).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'south', the agent moves by (dx=0, dy=-1).
  - The agent performs this exact sequence of actions: south -> south -> north -> north ->
  ↪  west -> west -> west -> south -> west -> north -> west -> east -> north -> west -> west
  ↪  -> north -> north -> south -> east -> north -> west -> east -> south -> south -> south
  ↪  -> north -> south -> south -> west -> north -> west -> east -> east -> east -> west ->
  ↪  west -> north -> north -> south -> east -> east -> east -> east -> north -> north ->
  ↪  north -> east -> north -> west -> south.

Apply the transition rules step by step starting from the initial position, following the
↪  action sequence in order. Output the full trajectory as a list of 50 coordinates (initial
↪  position followed by each subsequent position), one per line, in the format '[x,y]'.
```

```
[MODEL OUTPUT]:
Let's process the action sequence step by step starting from the initial position (2, 7) on a
↪  5x8 toroidal grid. The grid wraps around at the edges, so moving off one edge brings the
↪  agent to the opposite edge.

### Initial position:
- (2, 7)

### Action sequence:
1. south
2. south
3. north
4. north
5. west
6. west
7. west
8. south
9. west
10. north
11. west
12. east
13. north
14. west
15. west
16. north
17. north
18. south
19. east
20. north
21. west
22. east
23. south
24. south
25. south
26. north
27. south
28. south
29. west
30. north
31. west
32. east
33. east
34. east
35. west
36. west
37. north
38. north
39. south
40. east
41. east
42. east
43. east
44. north
45. north
46. north
47. east
48. north
49. west
50. south

### Movement rules (simplified and corrected from the given rules):
- 'south': (dx=0, dy=-1)
- 'north': (dx=0, dy=1)
- 'west': (dx=-1, dy=0)
- 'east': (dx=1, dy=0)

### Trajectory:
Let's compute each step:

1. Start: (2, 7)
2. south: (2, 6)
3. south: (2, 5)
4. north: (2, 6)
5. north: (2, 7)
6. west: (1, 7)
7. west: (0, 7)
8. west: (4, 7) [wraps around from x=0 to x=4]
9. south: (4, 6)
10. west: (3, 6)
11. north: (3, 7)
12. west: (2, 7)
13. north: (2, 0) [wraps around from y=7 to y=0]
14. west: (1, 0)
```

15. west: (0, 0)
16. north: (0, 1)
17. north: (0, 2)
18. south: (0, 1)
19. east: (1, 1)
20. north: (1, 2)
21. west: (0, 2)
22. east: (1, 2)
23. south: (1, 1)
24. south: (1, 0)
25. south: (1, 7) [wraps around from y=0 to y=7]
26. north: (1, 0)
27. south: (1, 7) [wraps around from y=0 to y=7]
28. south: (1, 6)
29. west: (0, 6)
30. north: (0, 7)
31. west: (4, 7) [wraps around from x=0 to x=4]
32. east: (0, 7) [wraps around from x=4 to x=0]
33. east: (1, 7)
34. east: (2, 7)
35. west: (1, 7)
36. west: (0, 7)
37. north: (0, 0) [wraps around from y=7 to y=0]
38. north: (0, 1)
39. south: (0, 0)
40. east: (1, 0)
41. east: (2, 0)
42. east: (3, 0)
43. east: (4, 0)
44. north: (4, 1)
45. north: (4, 2)
46. north: (4, 3)
47. east: (0, 3) [wraps around from x=4 to x=0]
48. north: (0, 4)
49. west: (4, 4) [wraps around from x=0 to x=4]
50. south: (4, 3)

Full trajectory:
```
[2,7]
[2,6]
[2,5]
[2,6]
[2,7]
[1,7]
[0,7]
[4,7]
[4,6]
[3,6]
[3,7]
[2,7]
[2,0]
[1,0]
[0,0]
[0,1]
[0,2]
[0,1]
[1,1]
[1,2]
[0,2]
[1,2]
[1,1]
[1,0]
[1,7]
[1,0]
[1,7]
[1,6]
[0,6]
[0,7]
[4,7]
[0,7]
[1,7]
[2,7]
[1,7]
[0,7]
[0,0]
[0,1]
[0,0]
[1,0]
[2,0]
[3,0]
[4,0]
[4,1]
```

```
[4,2]
[4,3]
[0,3]
[0,4]
[4,4]
[4,3]
```

```
[GROUND TRUTH EXPECTED]:
[2,7]
[2,6]
[2,5]
[2,6]
[2,7]
[1,7]
[0,7]
[4,7]
[4,6]
[3,6]
[3,7]
[2,7]
[3,7]
[3,0]
[2,0]
[1,0]
[1,1]
[1,2]
[1,1]
[2,1]
[2,2]
[1,2]
[2,2]
[2,1]
[2,0]
[2,7]
[2,0]
[2,7]
[2,6]
[1,6]
[1,7]
[0,7]
[1,7]
[2,7]
[3,7]
[2,7]
[1,7]
[1,0]
[1,1]
[1,0]
[2,0]
[3,0]
[4,0]
[0,0]
[0,1]
[0,2]
[0,3]
[1,3]
[1,4]
[0,4]
[0,3]


=== TIER 1 ===

[EXACT SENT PROMPT]:
Rules:
  - The world is a grid with width 10 and height 9. Coordinates are (x, y) with x in [0, 9]
  ↪  and y in [0, 8].
  - The grid is toroidal: movement off one edge wraps to the opposite edge.
  - The agent starts at position (8, 3).
  - Transition rules:
  -   - When the action is 'rjlox', the agent moves by (dx=0, dy=1).
  -   - When the action is 'wldvoath', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'ewxzxykn', the agent moves by (dx=1, dy=0).
  -   - When the action is 'wldvoath', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'pjyo', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'pjyo', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'ewxzxykn', the agent moves by (dx=1, dy=0).
  -   - When the action is 'ewxzxykn', the agent moves by (dx=1, dy=0).
  -   - When the action is 'wldvoath', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'rjlox', the agent moves by (dx=0, dy=1).
  -   - When the action is 'pjyo', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'wldvoath', the agent moves by (dx=-1, dy=0).
```

```
  -   - When the action is 'wldvoath', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'wldvoath', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'pjyo', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'wldvoath', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'wldvoath', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'ewxzxykn', the agent moves by (dx=1, dy=0).
  -   - When the action is 'pjyo', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'wldvoath', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'rjlox', the agent moves by (dx=0, dy=1).
  -   - When the action is 'rjlox', the agent moves by (dx=0, dy=1).
  -   - When the action is 'rjlox', the agent moves by (dx=0, dy=1).
  -   - When the action is 'pjyo', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'pjyo', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'ewxzxykn', the agent moves by (dx=1, dy=0).
  -   - When the action is 'wldvoath', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'pjyo', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'ewxzxykn', the agent moves by (dx=1, dy=0).
  -   - When the action is 'rjlox', the agent moves by (dx=0, dy=1).
  -   - When the action is 'rjlox', the agent moves by (dx=0, dy=1).
  -   - When the action is 'rjlox', the agent moves by (dx=0, dy=1).
  -   - When the action is 'rjlox', the agent moves by (dx=0, dy=1).
  -   - When the action is 'rjlox', the agent moves by (dx=0, dy=1).
  -   - When the action is 'ewxzxykn', the agent moves by (dx=1, dy=0).
  -   - When the action is 'wldvoath', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'pjyo', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'pjyo', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'wldvoath', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'wldvoath', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'wldvoath', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'ewxzxykn', the agent moves by (dx=1, dy=0).
  -   - When the action is 'ewxzxykn', the agent moves by (dx=1, dy=0).
  -   - When the action is 'rjlox', the agent moves by (dx=0, dy=1).
  -   - When the action is 'pjyo', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'pjyo', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'pjyo', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'rjlox', the agent moves by (dx=0, dy=1).
  -   - When the action is 'ewxzxykn', the agent moves by (dx=1, dy=0).
  -   - When the action is 'wldvoath', the agent moves by (dx=-1, dy=0).
  - The agent performs this exact sequence of actions: rjlox -> wldvoath -> ewxzxykn ->
  ↪  wldvoath -> pjyo -> pjyo -> ewxzxykn -> ewxzxykn -> wldvoath -> rjlox -> pjyo ->
  ↪  wldvoath -> wldvoath -> wldvoath -> pjyo -> wldvoath -> wldvoath -> ewxzxykn -> pjyo ->
  ↪  wldvoath -> rjlox -> rjlox -> rjlox -> pjyo -> pjyo -> ewxzxykn -> wldvoath -> pjyo ->
  ↪  ewxzxykn -> rjlox -> rjlox -> rjlox -> rjlox -> rjlox -> ewxzxykn -> wldvoath -> pjyo ->
  ↪  pjyo -> wldvoath -> wldvoath -> wldvoath -> ewxzxykn -> ewxzxykn -> rjlox -> pjyo ->
  ↪  pjyo -> pjyo -> rjlox -> ewxzxykn -> wldvoath.

Apply the transition rules step by step starting from the initial position, following the
↪  action sequence in order. Output the full trajectory as a list of 50 coordinates (initial
↪  position followed by each subsequent position), one per line, in the format '[x,y]'.

[MODEL OUTPUT]:
Let's process the action sequence step by step starting from the initial position (8, 3). The
↪  grid is toroidal, so coordinates wrap around when they go out of bounds (e.g., x = -1
↪  becomes 9, x = 10 becomes 0, y = -1 becomes 8, y = 9 becomes 0).

### Initial position:
- (8, 3)

### Action sequence and resulting positions:
1. 'rjlox' -> (dx=0, dy=1) -> (8, 3 + 1) = (8, 4)
2. 'wldvoath' -> (dx=-1, dy=0) -> (8 - 1, 4) = (7, 4)
3. 'ewxzxykn' -> (dx=1, dy=0) -> (7 + 1, 4) = (8, 4)
4. 'wldvoath' -> (dx=-1, dy=0) -> (8 - 1, 4) = (7, 4)
5. 'pjyo' -> (dx=0, dy=-1) -> (7, 4 - 1) = (7, 3)
6. 'pjyo' -> (dx=0, dy=-1) -> (7, 3 - 1) = (7, 2)
7. 'ewxzxykn' -> (dx=1, dy=0) -> (7 + 1, 2) = (8, 2)
8. 'ewxzxykn' -> (dx=1, dy=0) -> (8 + 1, 2) = (9, 2)
9. 'wldvoath' -> (dx=-1, dy=0) -> (9 - 1, 2) = (8, 2)
10. 'rjlox' -> (dx=0, dy=1) -> (8, 2 + 1) = (8, 3)
11. 'pjyo' -> (dx=0, dy=-1) -> (8, 3 - 1) = (8, 2)
12. 'wldvoath' -> (dx=-1, dy=0) -> (8 - 1, 2) = (7, 2)
13. 'wldvoath' -> (dx=-1, dy=0) -> (7 - 1, 2) = (6, 2)
14. 'wldvoath' -> (dx=-1, dy=0) -> (6 - 1, 2) = (5, 2)
15. 'pjyo' -> (dx=0, dy=-1) -> (5, 2 - 1) = (5, 1)
16. 'wldvoath' -> (dx=-1, dy=0) -> (5 - 1, 1) = (4, 1)
17. 'wldvoath' -> (dx=-1, dy=0) -> (4 - 1, 1) = (3, 1)
18. 'ewxzxykn' -> (dx=1, dy=0) -> (3 + 1, 1) = (4, 1)
19. 'pjyo' -> (dx=0, dy=-1) -> (4, 1 - 1) = (4, 0)
20. 'wldvoath' -> (dx=-1, dy=0) -> (4 - 1, 0) = (3, 0)
21. 'rjlox' -> (dx=0, dy=1) -> (3, 0 + 1) = (3, 1)
22. 'rjlox' -> (dx=0, dy=1) -> (3, 1 + 1) = (3, 2)
23. 'rjlox' -> (dx=0, dy=1) -> (3, 2 + 1) = (3, 3)
24. 'pjyo' -> (dx=0, dy=-1) -> (3, 3 - 1) = (3, 2)
```

```
25. 'pjyo' -> (dx=0, dy=-1) -> (3, 2 - 1) = (3, 1)
26. 'ewxzxykn' -> (dx=1, dy=0) -> (3 + 1, 1) = (4, 1)
27. 'wldvoath' -> (dx=-1, dy=0) -> (4 - 1, 1) = (3, 1)
28. 'pjyo' -> (dx=0, dy=-1) -> (3, 1 - 1) = (3, 0)
29. 'ewxzxykn' -> (dx=1, dy=0) -> (3 + 1, 0) = (4, 0)
30. 'rjlox' -> (dx=0, dy=1) -> (4, 0 + 1) = (4, 1)
31. 'rjlox' -> (dx=0, dy=1) -> (4, 1 + 1) = (4, 2)
32. 'rjlox' -> (dx=0, dy=1) -> (4, 2 + 1) = (4, 3)
33. 'rjlox' -> (dx=0, dy=1) -> (4, 3 + 1) = (4, 4)
34. 'rjlox' -> (dx=0, dy=1) -> (4, 4 + 1) = (4, 5)
35. 'ewxzxykn' -> (dx=1, dy=0) -> (4 + 1, 5) = (5, 5)
36. 'wldvoath' -> (dx=-1, dy=0) -> (5 - 1, 5) = (4, 5)
37. 'pjyo' -> (dx=0, dy=-1) -> (4, 5 - 1) = (4, 4)
38. 'pjyo' -> (dx=0, dy=-1) -> (4, 4 - 1) = (4, 3)
39. 'wldvoath' -> (dx=-1, dy=0) -> (4 - 1, 3) = (3, 3)
40. 'wldvoath' -> (dx=-1, dy=0) -> (3 - 1, 3) = (2, 3)
41. 'wldvoath' -> (dx=-1, dy=0) -> (2 - 1, 3) = (1, 3)
42. 'ewxzxykn' -> (dx=1, dy=0) -> (1 + 1, 3) = (2, 3)
43. 'ewxzxykn' -> (dx=1, dy=0) -> (2 + 1, 3) = (3, 3)
44. 'rjlox' -> (dx=0, dy=1) -> (3, 3 + 1) = (3, 4)
45. 'pjyo' -> (dx=0, dy=-1) -> (3, 4 - 1) = (3, 3)
46. 'pjyo' -> (dx=0, dy=-1) -> (3, 3 - 1) = (3, 2)
47. 'pjyo' -> (dx=0, dy=-1) -> (3, 2 - 1) = (3, 1)
48. 'rjlox' -> (dx=0, dy=1) -> (3, 1 + 1) = (3, 2)
49. 'ewxzxykn' -> (dx=1, dy=0) -> (3 + 1, 2) = (4, 2)
50. 'wldvoath' -> (dx=-1, dy=0) -> (4 - 1, 2) = (3, 2)

### Full trajectory:
```
[8,3]
[8,4]
[7,4]
[8,4]
[7,4]
[7,3]
[7,2]
[8,2]
[9,2]
[8,2]
[8,3]
[8,2]
[7,2]
[6,2]
[5,2]
[5,1]
[4,1]
[3,1]
[4,1]
[4,0]
[3,0]
[3,1]
[3,2]
[3,3]
[3,2]
[3,1]
[4,1]
[3,1]
[3,0]
[4,0]
[4,1]
[4,2]
[4,3]
[4,4]
[4,5]
[5,5]
[4,5]
[4,4]
[4,3]
[3,3]
[2,3]
[1,3]
[2,3]
[3,3]
[3,4]
[3,3]
[3,2]
[3,1]
[3,2]
[4,2]
[3,2]
```

[GROUND TRUTH EXPECTED]:
```

```
[8,3]
[8,4]
[7,4]
[8,4]
[7,4]
[7,3]
[7,2]
[8,2]
[9,2]
[8,2]
[8,3]
[8,2]
[7,2]
[6,2]
[5,2]
[5,1]
[4,1]
[3,1]
[4,1]
[4,0]
[3,0]
[3,1]
[3,2]
[3,3]
[3,2]
[3,1]
[4,1]
[3,1]
[3,0]
[4,0]
[4,1]
[4,2]
[4,3]
[4,4]
[4,5]
[5,5]
[4,5]
[4,4]
[4,3]
[3,3]
[2,3]
[1,3]
[2,3]
[3,3]
[3,4]
[3,3]
[3,2]
[3,1]
[3,2]
[4,2]
[3,2]


=== TIER 2 ===

[EXACT SENT PROMPT]:
Rules:
  - The world is a grid with width 8 and height 11. Coordinates are (x, y) with x in [0, 7]
  ↪  and y in [0, 10].
  - The grid is toroidal: movement off one edge wraps to the opposite edge.
  - The agent starts at position (6, 8).
  - Transition rules:
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'north', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'south', the agent moves by (dx=0, dy=1).
  -   - When the action is 'south', the agent moves by (dx=0, dy=1).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'south', the agent moves by (dx=0, dy=1).
  -   - When the action is 'south', the agent moves by (dx=0, dy=1).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'north', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'north', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'north', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'south', the agent moves by (dx=0, dy=1).
  -   - When the action is 'north', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
```

```
  -   - When the action is 'south', the agent moves by (dx=0, dy=1).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'north', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'north', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'south', the agent moves by (dx=0, dy=1).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'south', the agent moves by (dx=0, dy=1).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'south', the agent moves by (dx=0, dy=1).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'south', the agent moves by (dx=0, dy=1).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'north', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'west', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'east', the agent moves by (dx=1, dy=0).
  -   - When the action is 'north', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'south', the agent moves by (dx=0, dy=1).
  -   - When the action is 'north', the agent moves by (dx=0, dy=-1).
  - The agent performs this exact sequence of actions: east -> north -> east -> east -> east
  ↪  -> east -> south -> south -> east -> west -> south -> south -> east -> east -> north ->
  ↪  north -> north -> west -> south -> north -> east -> south -> east -> east -> west ->
  ↪  north -> west -> north -> south -> west -> west -> west -> south -> west -> south ->
  ↪  east -> west -> west -> south -> west -> east -> west -> north -> west -> west -> west
  ↪  -> east -> north -> south -> north.

Apply the transition rules step by step starting from the initial position, following the
↪  action sequence in order. Output the full trajectory as a list of 50 coordinates (initial
↪  position followed by each subsequent position), one per line, in the format '[x,y]'.

[MODEL OUTPUT]:
Let's compute the trajectory step by step starting from the initial position (6, 8) and
↪  following the given sequence of actions. The grid is toroidal, so movements wrap around
↪  when they go out of bounds (e.g., x = -1 becomes 7, x = 8 becomes 0, y = -1 becomes 10, y
↪  = 11 becomes 0).

### Initial position:
- (6, 8)

### Action sequence and trajectory:
Here is the full trajectory after each action:

1. Start: [6, 8]
2. east: (6+1, 8) = [7, 8]
3. north: (7, 8-1) = [7, 7]
4. east: (7+1, 7) = [0, 7] (wraps around x)
5. east: (0+1, 7) = [1, 7]
6. east: (1+1, 7) = [2, 7]
7. east: (2+1, 7) = [3, 7]
8. south: (3, 7+1) = [3, 8]
9. south: (3, 8+1) = [3, 9]
10. east: (3+1, 9) = [4, 9]
11. west: (4-1, 9) = [3, 9]
12. south: (3, 9+1) = [3, 10]
13. south: (3, 10+1) = [3, 0] (wraps around y)
14. east: (3+1, 0) = [4, 0]
15. east: (4+1, 0) = [5, 0]
16. north: (5, 0-1) = [5, 10] (wraps around y)
17. north: (5, 10-1) = [5, 9]
18. north: (5, 9-1) = [5, 8]
19. west: (5-1, 8) = [4, 8]
20. south: (4, 8+1) = [4, 9]
21. north: (4, 9-1) = [4, 8]
22. east: (4+1, 8) = [5, 8]
23. south: (5, 8+1) = [5, 9]
24. east: (5+1, 9) = [6, 9]
25. east: (6+1, 9) = [7, 9]
26. west: (7-1, 9) = [6, 9]
27. north: (6, 9-1) = [6, 8]
28. west: (6-1, 8) = [5, 8]
29. north: (5, 8-1) = [5, 7]
30. south: (5, 7+1) = [5, 8]
31. west: (5-1, 8) = [4, 8]
```

````
32. west: (4-1, 8) = [3, 8]
33. west: (3-1, 8) = [2, 8]
34. south: (2, 8+1) = [2, 9]
35. west: (2-1, 9) = [1, 9]
36. south: (1, 9+1) = [1, 10]
37. east: (1+1, 10) = [2, 10]
38. west: (2-1, 10) = [1, 10]
39. west: (1-1, 10) = [0, 10]
40. south: (0, 10+1) = [0, 0] (wraps around y)
41. west: (0-1, 0) = [7, 0] (wraps around x)
42. east: (7+1, 0) = [0, 0]
43. west: (0-1, 0) = [7, 0]
44. north: (7, 0-1) = [7, 10] (wraps around y)
45. west: (7-1, 10) = [6, 10]
46. west: (6-1, 10) = [5, 10]
47. west: (5-1, 10) = [4, 10]
48. east: (4+1, 10) = [5, 10]
49. north: (5, 10-1) = [5, 9]
50. south: (5, 9+1) = [5, 10]

### Full trajectory:
```
[6, 8]
[7, 8]
[7, 7]
[0, 7]
[1, 7]
[2, 7]
[3, 7]
[3, 8]
[3, 9]
[4, 9]
[3, 9]
[3, 10]
[3, 0]
[4, 0]
[5, 0]
[5, 10]
[5, 9]
[5, 8]
[4, 8]
[4, 9]
[4, 8]
[5, 8]
[5, 9]
[6, 9]
[7, 9]
[6, 9]
[6, 8]
[5, 8]
[5, 7]
[5, 8]
[4, 8]
[3, 8]
[2, 8]
[2, 9]
[1, 9]
[1, 10]
[2, 10]
[1, 10]
[0, 10]
[0, 0]
[7, 0]
[0, 0]
[7, 0]
[7, 10]
[6, 10]
[5, 10]
[4, 10]
[5, 10]
[5, 9]
[5, 10]
```

[GROUND TRUTH EXPECTED]:
[6,8]
[7,8]
[7,7]
[0,7]
[1,7]
[2,7]
[3,7]
[3,8]
````

```
[3,9]
[4,9]
[3,9]
[3,10]
[3,0]
[4,0]
[5,0]
[5,10]
[5,9]
[5,8]
[4,8]
[4,9]
[4,8]
[5,8]
[5,9]
[6,9]
[7,9]
[6,9]
[6,8]
[5,8]
[5,7]
[5,8]
[4,8]
[3,8]
[2,8]
[2,9]
[1,9]
[1,10]
[2,10]
[1,10]
[0,10]
[0,0]
[7,0]
[0,0]
[7,0]
[7,10]
[6,10]
[5,10]
[4,10]
[5,10]
[5,9]
[5,10]
[5,9]


=== TIER 3 ===

[EXACT SENT PROMPT]:
Rules:
  - The world is a grid with width 8 and height 6. Coordinates are (x, y) with x in [0, 7] and
  ↪  y in [0, 5].
  - The grid is toroidal: movement off one edge wraps to the opposite edge.
  - The agent starts at position (7, 0).
  - Transition rules:
  -   - When the action is 'east', the agent moves by (dx=0, dy=1).
  -   - When the action is 'east', the agent moves by (dx=0, dy=1).
  -   - When the action is 'south', the agent moves by (dx=1, dy=0).
  -   - When the action is 'north', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'north', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'east', the agent moves by (dx=0, dy=1).
  -   - When the action is 'north', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'east', the agent moves by (dx=0, dy=1).
  -   - When the action is 'east', the agent moves by (dx=0, dy=1).
  -   - When the action is 'east', the agent moves by (dx=0, dy=1).
  -   - When the action is 'south', the agent moves by (dx=1, dy=0).
  -   - When the action is 'east', the agent moves by (dx=0, dy=1).
  -   - When the action is 'east', the agent moves by (dx=0, dy=1).
  -   - When the action is 'west', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'east', the agent moves by (dx=0, dy=1).
  -   - When the action is 'west', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'north', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'west', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'south', the agent moves by (dx=1, dy=0).
  -   - When the action is 'south', the agent moves by (dx=1, dy=0).
  -   - When the action is 'east', the agent moves by (dx=0, dy=1).
  -   - When the action is 'east', the agent moves by (dx=0, dy=1).
  -   - When the action is 'east', the agent moves by (dx=0, dy=1).
  -   - When the action is 'north', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'north', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'north', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'north', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'west', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'west', the agent moves by (dx=0, dy=-1).
```

```
  -   - When the action is 'south', the agent moves by (dx=1, dy=0).
  -   - When the action is 'west', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'south', the agent moves by (dx=1, dy=0).
  -   - When the action is 'north', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'east', the agent moves by (dx=0, dy=1).
  -   - When the action is 'south', the agent moves by (dx=1, dy=0).
  -   - When the action is 'south', the agent moves by (dx=1, dy=0).
  -   - When the action is 'west', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'west', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'east', the agent moves by (dx=0, dy=1).
  -   - When the action is 'north', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'north', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'south', the agent moves by (dx=1, dy=0).
  -   - When the action is 'west', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'north', the agent moves by (dx=-1, dy=0).
  -   - When the action is 'west', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'west', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'west', the agent moves by (dx=0, dy=-1).
  -   - When the action is 'east', the agent moves by (dx=0, dy=1).
  -   - When the action is 'south', the agent moves by (dx=1, dy=0).
  -   - When the action is 'north', the agent moves by (dx=-1, dy=0).
  - The agent performs this exact sequence of actions: east -> east -> south -> north -> north
  ↪  -> east -> north -> east -> east -> east -> south -> east -> east -> west -> east ->
  ↪  west -> north -> west -> south -> south -> east -> east -> east -> north -> north ->
  ↪  north -> north -> west -> west -> south -> west -> south -> north -> east -> south ->
  ↪  south -> west -> west -> east -> north -> north -> south -> west -> north -> west ->
  ↪  west -> west -> east -> south -> north.

Apply the transition rules step by step starting from the initial position, following the
↪  action sequence in order. Output the full trajectory as a list of 50 coordinates (initial
↪  position followed by each subsequent position), one per line, in the format '[x,y]'.

[MODEL OUTPUT]:
Let's carefully follow the action sequence step by step, starting from the initial position
↪  `(7, 0)` on the toroidal grid (8x6). The grid wraps around at the edges, so moving off one
↪  edge brings you to the opposite edge.

### Initial position:
- `(7, 0)`

### Action sequence:
1. east
2. east
3. south
4. north
5. north
6. east
7. north
8. east
9. east
10. east
11. south
12. east
13. east
14. west
15. east
16. west
17. north
18. west
19. south
20. south
21. east
22. east
23. east
24. north
25. north
26. north
27. north
28. west
29. west
30. south
31. west
32. south
33. north
34. east
35. south
36. south
37. west
38. west
39. east
40. north
41. north
42. south
```

````
43. west
44. north
45. west
46. west
47. west
48. east
49. south
50. north

### Movement rules (corrected from the original, which had duplicates and inconsistencies):
- 'east': `(dx=0, dy=1)` (move up in y)
- 'west': `(dx=0, dy=-1)` (move down in y)
- 'south': `(dx=1, dy=0)` (move right in x)
- 'north': `(dx=-1, dy=0)` (move left in x)

### Trajectory:
Let's compute each step:

1. Start: `(7, 0)`
2. east: `(7, 0) + (0, 1) = (7, 1)`
3. east: `(7, 1) + (0, 1) = (7, 2)`
4. south: `(7, 2) + (1, 0) = (0, 2)` (wraps around x)
5. north: `(0, 2) + (-1, 0) = (7, 2)` (wraps around x)
6. north: `(7, 2) + (-1, 0) = (6, 2)`
7. east: `(6, 2) + (0, 1) = (6, 3)`
8. north: `(6, 3) + (-1, 0) = (5, 3)`
9. east: `(5, 3) + (0, 1) = (5, 4)`
10. east: `(5, 4) + (0, 1) = (5, 5)`
11. east: `(5, 5) + (0, 1) = (5, 0)` (wraps around y)
12. south: `(5, 0) + (1, 0) = (6, 0)`
13. east: `(6, 0) + (0, 1) = (6, 1)`
14. east: `(6, 1) + (0, 1) = (6, 2)`
15. west: `(6, 2) + (0, -1) = (6, 1)`
16. east: `(6, 1) + (0, 1) = (6, 2)`
17. west: `(6, 2) + (0, -1) = (6, 1)`
18. north: `(6, 1) + (-1, 0) = (5, 1)`
19. west: `(5, 1) + (0, -1) = (5, 0)`
20. south: `(5, 0) + (1, 0) = (6, 0)`
21. south: `(6, 0) + (1, 0) = (7, 0)`
22. east: `(7, 0) + (0, 1) = (7, 1)`
23. east: `(7, 1) + (0, 1) = (7, 2)`
24. east: `(7, 2) + (0, 1) = (7, 3)`
25. north: `(7, 3) + (-1, 0) = (6, 3)`
26. north: `(6, 3) + (-1, 0) = (5, 3)`
27. north: `(5, 3) + (-1, 0) = (4, 3)`
28. north: `(4, 3) + (-1, 0) = (3, 3)`
29. west: `(3, 3) + (0, -1) = (3, 2)`
30. west: `(3, 2) + (0, -1) = (3, 1)`
31. south: `(3, 1) + (1, 0) = (4, 1)`
32. west: `(4, 1) + (0, -1) = (4, 0)`
33. south: `(4, 0) + (1, 0) = (5, 0)`
34. north: `(5, 0) + (-1, 0) = (4, 0)`
35. east: `(4, 0) + (0, 1) = (4, 1)`
36. south: `(4, 1) + (1, 0) = (5, 1)`
37. south: `(5, 1) + (1, 0) = (6, 1)`
38. west: `(6, 1) + (0, -1) = (6, 0)`
39. west: `(6, 0) + (0, -1) = (6, 5)` (wraps around y)
40. east: `(6, 5) + (0, 1) = (6, 0)`
41. north: `(6, 0) + (-1, 0) = (5, 0)`
42. north: `(5, 0) + (-1, 0) = (4, 0)`
43. south: `(4, 0) + (1, 0) = (5, 0)`
44. west: `(5, 0) + (0, -1) = (5, 5)`
45. north: `(5, 5) + (-1, 0) = (4, 5)`
46. west: `(4, 5) + (0, -1) = (4, 4)`
47. west: `(4, 4) + (0, -1) = (4, 3)`
48. west: `(4, 3) + (0, -1) = (4, 2)`
49. east: `(4, 2) + (0, 1) = (4, 3)`
50. south: `(4, 3) + (1, 0) = (5, 3)`
51. north: `(5, 3) + (-1, 0) = (4, 3)`

### Final trajectory (50 steps, initial + 49 actions):
```
[7,0]
[7,1]
[7,2]
[0,2]
[7,2]
[6,2]
[6,3]
[5,3]
[5,4]
[5,5]
[5,0]
````
````

```
[6,0]
[6,1]
[6,2]
[6,1]
[6,2]
[6,1]
[5,1]
[5,0]
[6,0]
[7,0]
[7,1]
[7,2]
[7,3]
[6,3]
[5,3]
[4,3]
[3,3]
[3,2]
[3,1]
[4,1]
[4,0]
[5,0]
[4,0]
[4,1]
[5,1]
[6,1]
[6,0]
[6,5]
[6,0]
[5,0]
[4,0]
[5,0]
[5,5]
[4,5]
[4,4]
[4,3]
[4,2]
[4,3]
[5,3]
[4,3]
```

```
[GROUND TRUTH EXPECTED]:
[7,0]
[7,1]
[7,2]
[0,2]
[7,2]
[6,2]
[6,3]
[5,3]
[5,4]
[5,5]
[5,0]
[6,0]
[6,1]
[6,2]
[6,1]
[6,2]
[6,1]
[5,1]
[5,0]
[6,0]
[7,0]
[7,1]
[7,2]
[7,3]
[6,3]
[5,3]
[4,3]
[3,3]
[3,2]
[3,1]
[4,1]
[4,0]
[5,0]
[4,0]
[4,1]
[5,1]
[6,1]
[6,0]
[6,5]
[6,0]
```

```
[5,0]
[4,0]
[5,0]
[5,5]
[4,5]
[4,4]
[4,3]
[4,2]
[4,3]
[5,3]
[4,3]
```